LEARN · VERIFY — OFFLINE STUDY BOOKLET

Working at Wider Scope

Past a single team, the first thing that fails is not your skill — it is your view. Here is why depth costs you perspective, how to read the ground you have to cross and where it is all supposed to be going, how an overwhelming project becomes a tractable one, and why at this scope how you work becomes part of what you produce.

Print double-sided, short-edge bind, A4 · 100% scale (no “fit to page”).

Answer key starts at Section 9 — keep a sheet of paper over it.

How to use this booklet

The loop, per section

1. Read the plain-version box first. If it doesn't land, the rest won't either.
2. Read the teaching. Say each boundary out loud: “X is *this*, its look-alike Y is *that*.”
3. Cover the page. Write the answer to **Q1** in the blank lines, in your own words, in full sentences. **Then** check the key.
4. Score yourself on completeness. Then do **Q2** and note whether it beat Q1.
5. Only then move on.

> **THE RULE THAT DOES MOST OF THE WORK**
>
> If you find yourself thinking “yes, I know this” — that is *recognition*, and it is not learning. The only evidence that counts is a written answer produced with the page covered.

> **WHAT THIS SUBJECT ASKS OF YOU THAT OTHERS DON'T**
>
> Most engineering subjects have a right answer sitting somewhere in the system. This one mostly does not. It is about **the humans around the system**, and it keeps handing you claims that sound like opinion until you notice they are load-bearing: that being right is less than half the battle, that the same depth which makes you good makes you mis-rank your own work, that a thing nobody is paid to do is exactly the thing you should be doing. Two habits keep you honest here. **First, every claim in this booklet has a stated mechanism** — a reason it works — and if you can only recall the advice and not the mechanism, you have memorised a slogan. **Second, resist agreeing.** Agreement is the cheapest possible response to material like this and it teaches you nothing. Ask instead what each idea predicts: what you would expect to see in an organisation that had it, and what you would expect to see in one that did not.

The spacing schedule

Do a part, then review it after **1 day**, **3 days**, **7 days**, **16 days** and **35 days**. A review is not rereading: it is covering the page and answering the questions in Section 10's tracker. On a fail, that row goes back to a 1-day interval.

What's in here

SECTION	CONTENTS
1	Learning goal, the core model in one paragraph, and the four-part roadmap.
2–5	One part each, always in the same order: plain version → everyday analogy → term table with boundaries → trade-offs → concept-boundary box → anchor sketch → plain/expert phrasing → two written questions on cases that are not in the teaching → a 2×2 → a carry-out rule.
6	The whole subject as one concept sketch, with a question on every node, plus the master 2×2.
7	Symptom → diagnosis → move, and the reverse mapping: you see a practice, what is it for?
8	Numbers worth remembering, and the glossary.
9	Answer key for the eight written questions.
10	Spaced-review tracker and the final quiz.

SECTION 1

Learning goal & roadmap

LEARNING GOAL — WHAT YOU SHOULD BE ABLE TO DO WHEN YOU CLOSE THIS

Handed a scope larger than one team — a cross-team project, an initiative nobody has agreed to, an organisation you have to move — you can:

1. **Say what you cannot see and why**, naming which of the four risks of depth you are currently exhibiting, and run at least two of the techniques for looking at your own group from outside it.
2. **Describe your organisation's terrain** — where it sits on the seven cultural attributes, which of the named points of interest are in your way, where decisions actually get made and who the unofficial influencers are — and say what each fact changes about how you would introduce an idea.
3. **Take an overwhelming project and make it tractable**: gather the eight points of context, put roles and milestones in place, and drive it deliberately — explore before designing, write it down, and name which design pitfall a plan is falling into.
4. **Account for the influence you have passively**, naming the four attributes you are modelling whether you mean to or not, and say what each one predicts about the behaviour of the people watching you.

The core model in one paragraph

One thing changes when your scope grows past a single team, and everything else here follows from it: **the constraint stops being what you can build and becomes what you can see**. Depth in a domain is bought with time inside it, and time inside it costs perspective — your group's problems start to feel more important than they are, other people's start to feel simpler than they are, the broken things you have lived with stop registering at all, and the reason for the work quietly detaches from the work. So the first job is to get an outside view of your own position. The second is that a position is not enough to navigate by: you need the terrain — how your organisation actually behaves, where the chasms and gatekeepers and unwinnable fights are, where decisions are really made — and you need the destination, because a team that does not know where it is going cannot be trusted to choose its own route. Those two together are what turn an overwhelming project into a tractable one: the overwhelm at the start of a big project is an *information* problem, not a character problem, and it is answered by building context, setting up structures, and then driving the thing rather than letting it happen. And there is a sting in the tail. At this scope your work is checked less and copied more; people stop guiding you and start looking to you for guidance. That makes **how you work part of what you produce** — your standards, your handling of your own mistakes, your willingness to say “I don't know”, your calm — and it is the one output you cannot decline to ship.

TENTH-GRADER VERSION — THE WHOLE THING AT ONCE

- The longer you work on one thing, the better you get at it and the worse you get at judging how much it matters.
- Four things go wrong: you rank badly, you assume other people's work is easy, you stop noticing what's broken near you, and you forget why you started.
- So you deliberately look at your own team the way a stranger would.
- Knowing where you are isn't enough. You also need to know what the ground is like — where the arguments are, who really decides things — and where everybody is trying to get to.
- Big projects feel impossible at the start. That feeling is just missing information, and there's a list of what to go and find.
- Then you have to actually steer: explore before you design, write it down, and expect something to go wrong.
- And here's the catch: at this point people copy you. Sloppy work makes a sloppy team, without you ever telling anyone to be sloppy.

The four parts, and why they're in this order

PART	WHAT IT COVERS	WHY HERE
1 What you cannot see from here	Why context has to be gathered deliberately; the skill of paying attention and the separate problem of access; the four risks that come with depth — prioritising badly, losing empathy, tuning out the background noise, forgetting what the work is for; and five techniques for seeing bigger.	First, because it is the failure you cannot detect from inside. Every later part depends on being able to look at your own position honestly.
2 The terrain and the destination	Organisations as terrain; the seven cultural attributes; the three-way typology of how organisations handle information; chasms, fortresses, disputed territory, uncrossable deserts, paved roads and goat tracks; where decisions happen and the shadow org chart; keeping the map fresh; bridging; and the five costs of having no destination.	Position without terrain is not navigation. This part is what converts “I am right” into “this can actually happen here”, and it supplies the destination that Part 3's project is a step toward.
3 Making a huge project tractable	Why big projects are hard (ambiguity, not technology); the overwhelm and five things that reduce it; the eight points of context; roles, recruiting, scope and milestones, estimation, logistics; and driving — exploring, clarifying, designing, the nine design pitfalls, communicating status, navigating obstacles.	Third, because it is the application. Every structure here is context made durable, and every driving technique is one of Part 1's or Part 2's failures caught early.
4 The influence you did not ask for	Passive influence and why values are what you do; the four attributes — being competent, being responsible, remembering the goal, looking ahead — and what each is made of, from admitting what you don't know to designing systems that can be decommissioned.	Last, because it is the part that runs while the other three are happening. It is not a chapter on ethics; it is a claim about causation — how you behave sets what the people watching you think normal is.

> *“Being right about a need for change is less than half the battle.”*

1

PART 1 OF 4

What you cannot see from here

Why depth costs perspective, the four specific ways it costs it, and how to get an outside view of your own work.

TENTH-GRADER VERSION

- When you start a new job you can't see much. Over time you learn your corner really well — and only your corner.
- Four bad things follow. You start thinking your team's problems are the important ones. You start thinking everyone else's job is easy. You stop noticing the broken things you walk past every day. And you forget what the work was supposed to be for.
- Notice that two of those are opposites: your stuff feels too important, *and* your stuff stops registering. Both come from being in one place too long.
- Knowing things needs two separate things: noticing (a skill) and being allowed in the room (an opportunity). Being good at one doesn't fix the other.
- New people can always see the problems, because they weren't here while things slowly got worse. Your job is to keep that view after you stop being new.
- And check what the company actually needs right now. Some of what it needs is so obvious nobody says it out loud — until it's in danger.

THE EVERYDAY VERSION

Two people walk the same country lane. One has lived there for years and can name the pine marten from a print in the mud, can tell you which of the grasses is peppery and worth eating, and knows where the wild strawberries come up. The other sees an oak tree and some flowers and thinks they are seeing everything there is. The difference is not eyesight. It is that one of them learned what to look for and got into the habit of paying attention — and it can be learned. Now change the picture slightly. The person who has lived there for years walks past the collapsed gate every day and no longer sees it; the visitor notices it in the first ten seconds and asks what happened. That is the whole of this part. **Familiarity buys you detail and charges you the ability to see the obvious**, and both halves of that bargain are permanent unless you do something deliberate about them.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
The fog of war	The parts of your organisation's picture that are **obscured rather than absent**. The information already exists; you have not explored it yet.	Starting a new job, most of the big picture is completely unknown to you. As you scout around you clear fog and learn what surrounds you. Some of it clears through everyday learning; the rest you must **deliberately set out to clear**.	vs ignorance you can fix by trying harder at your job. The distinction matters because it makes context-gathering a *task* with a plan, not a personality trait. It also fogs back up: as priorities change, parts of the map you had cleared stop being true.

The locator map	Your **place in the wider organisation and company** — what is along the borders of your scope, and how big your part of the world is compared with everywhere else.	The map a news broadcast throws up behind the presenter to remind you where a place is. Be honest about which of the projects you care about would show up on a company-wide map, and which you would not see unless you zoomed all the way in.	vs your scope, which you can state from your job description. Scope tells you what is inside; the locator map is specifically **what is outside**. You need it because **it is tricky to be objective about any work while you are deep inside it**.
Paying attention (the skill)	Being **alert to facts that affect your projects or organisation** — continually sifting information out of the noise around you.	An all-hands about a marketing push is a hint that traffic spikes you are not ready for are coming. Your director asks for something you have no time for, and you know which engineers are ready to stretch. Your database vanishes and you remember an email about network maintenance.	vs being well informed. This is a trainable habit: teach your brain to say “that's interesting” and retain the fact. If your memory is not naturally sticky for this, write the facts down purely to build the habit.
Access (the opportunity)	Being present for the **decisions and discussions that affect your work**. Noticing does not help if the information never comes near you.	There is a great deal you cannot ask for **unless you know it exists**. Beyond your daily flow of meetings and messages sits a second layer of decisions being made in rooms you are not in.	vs paying attention, which is the other half and does not substitute. Two people can be equally observant and have completely different pictures. Treat this as a separate problem with separate remedies — covered in Part 2.
Risk 1 — prioritising badly	When everyone around you cares about the same set of things, **it is easy to magnify the importance of those things**, and problems outside your group start to look simple or unimportant by comparison.	This is where **local maximum** decisions come from: the local maximum starts to feel really important. The more time you spend staring at your own group's problems, the more they seem special and unique and worthy of special, unique solutions.	And sometimes they are. But **it is unusual to find a problem that is genuinely brand new**: checking for prior art and preexisting solutions means less time reinventing wheels. The failure is not caring about your work; it is losing the exchange rate between your work and everyone else's.
Risk 2 — losing empathy	Overfocusing until you forget the rest of the world exists, or start treating other technology areas as trivial next to your own rich, nuanced domain.	Like **a fish-eye lens** that makes the thing in front of you huge and squeezes everything else to the periphery: “that problem they're solving is easy, I could solve it in a weekend.” It also shows up in incidents, where teams get absorbed in the interesting technical details and forget there are users waiting for the system to come back.	vs arrogance. It is a perspective artefact, and it silently shapes **the words you use, what you explain versus leave implicit, and the motivations you ascribe to other people**. That is why it is so difficult for engineers to communicate with nonengineers.
Risk 3 — tuning out the background noise	The exact opposite failure: **you stop noticing problems at all**.	Work around the same mucky configuration file or broken deploy process for months and you stop thinking of it as something to fix. You may also fail to notice that something merely annoying has slowly got worse — **a problem close to becoming a crisis that you cannot react to proportionately because you no longer see it**.	vs risk 1, which is its mirror image, and the pair is the reason perspective cannot be replaced by caring more. One says your things matter too much, the other says they have stopped registering — and **the same cause produces both**.

Risk 4 — forgetting what the work is for	Losing the connection between the project and the goal it was meant to serve.	Your group took on a project to solve a larger goal; the project is still running though the goal no longer matters or **has already been solved some other way**. Working only on your own part, you slip into a world where everyone does their little piece and **nobody feels responsible for the end result**.	vs poor prioritisation, which is a ranking error; this is a **severed link**. The stated stakes are wider than efficiency: you can lose sight of the ethics of what you are doing and find yourself on something you would not be comfortable with if you stepped back and looked at the whole picture.
The outsider view	Looking at your own group **as if you were not part of it**, and being honest about what you see.	**A new person can always see the problems.** They have no historical context, they were not around for the gradual change, and they are just seeing the raw situation as it is — free to ask “what's really happening here? is any of this working?” The questions to keep asking yourself: do your technical decisions only make sense to people who have forgotten there is a world outside your team? If you all stopped, how long before anyone noticed or cared?	Being new is the best opportunity you will get for this, but the point is to hold the perspective permanently. And it is **not a licence to be a jerk**: hindsight makes “this is terrible, why didn't they just…” cheap. Have humility and assume there are good reasons for things being as they are — **respect what came before**.
The echo chamber	A group in which everyone you meet holds the same set of opinions, so your beliefs are never actually tested.	Coming from years at the bottom of the stack, meeting product teams who move fast, take risks and treat features customers love as at least as important as rock-solid reliability, is a shock — and the debates make firmly held beliefs more nuanced.	The remedy is peers **outside** your group: build relationships with other senior engineers until you can speak the truth to each other without it being contentious. Treat them as a team you belong to, alongside your actual team. Include understanding **negative opinions other teams hold about your group**; seeing what is valid in them improves your work.
The objectives that are always true	The needs of the company that **are so obvious they are only stated if they are in danger** — continuing to exist, having enough money to pay everyone, having a good reputation.	The product should work. Customers should want to use it. Deploying it should not be painfully slow. These sit underneath the stated objectives and metrics, and you are expected to know the implicit goals as well as the explicit ones.	vs the published objectives, which change every quarter and are the only ones most people can name. Note the ranking changes over time: if customers are leaving because you lack core features, it is not the moment to push technical debt; if things are smooth and growth is coming, it is a good moment to make the foundations solid.

CONCEPT BOUNDARY — PERSPECTIVE IS AN ACCURACY PROBLEM, NOT A MODESTY PROBLEM

What it sounds like: a request to be humble about your team's work and take other people's more seriously — a matter of attitude, and one you can satisfy by being nice about it.
What it actually is: a claim about a measurable distortion. Time inside a domain reliably changes your estimate of how much things matter, in *both* directions at once — your group's concerns inflate, and the defects you live beside deflate to zero. Being generous about other teams does not correct either one, because the error is in the measuring instrument.
Why that matters practically: the remedies are all mechanical rather than attitudinal. Look at the org chart and see how much of it your group occupies. Ask a new person what they see. Go and talk to peers outside your group, and to people outside engineering. Ask what the business considers important right now, and what it needs so badly that nobody says it aloud. Find out how your customers judge you rather than how you judge you. Check whether the problem has been solved before.
The trap: concluding that your work does not matter. That is the same error with the sign flipped. The test that is offered is neither flattering nor humble but specific — **it is fine not to be working on the most important thing, but what you are doing should not be a waste of your time; if you cannot explain to yourself why what you are doing needs someone at your level, you may be doing the wrong thing**.

Anchor sketch — why the view goes, and how to get it back

Legend: bold = a named risk or technique **reversed = the fact everything else answers** **Q = cover the page and answer this**

```
THREE MAPS ALREADY EXIST IN YOUR ORG. they are OBSCURED,
not missing. clearing them is a task, not a talent.
     locator = where you are . topographic = the terrain
     treasure = where you are all going
                       |
KNOWING THINGS NEEDS TWO SEPARATE THINGS
     SKILL . paying attention. sift facts out of noise.
          train the brain to say "that's interesting".
     OPPORTUNITY . access. you cannot ask for what you
          do not know exists. (-> part 2, "the room")
     -> being good at one does NOT fix the other.
                       |
DEPTH IN A DOMAIN IS BOUGHT WITH TIME INSIDE IT,
AND TIME INSIDE IT IS PAID FOR IN PERSPECTIVE.
                       |
FOUR RISKS OF DEPTH
-----------------------------------------------------
PRIORITISING BADLY . everyone near you cares about the
     same things, so those things inflate. outside
     problems look simple. the LOCAL MAXIMUM feels big.
     -> almost no problem is genuinely new. check prior art.
LOSING EMPATHY . fish-eye lens. "I could do that in a
     weekend." shapes your words, what you leave implicit,
     and the motives you assign to others.
     -> also why incidents forget the waiting users.
TUNING OUT . the OPPOSITE failure. the mucky config,
     the broken deploy: you stop seeing them. a problem
     can approach crisis unnoticed.
     -> risks 1 and 3 are mirrors from ONE cause.
```

```
FORGETTING WHAT IT IS FOR . the goal died or was met
     elsewhere; the project runs on. everyone does their
     piece; nobody owns the end result.
     -> at the limit you lose sight of the ethics too.
-----------------------------------------------------
FIVE WAYS TO SEE BIGGER
OUTSIDER VIEW . a new person can always see the
     problems -- no history, no gradual change.
     but being new is NOT a licence to be a jerk:
     assume good reasons. respect what came before.
ESCAPE THE ECHO CHAMBER . peers OUTSIDE your group are
     a team you belong to. hear the criticism of your
     own group. go beyond engineering entirely.
WHAT IS ACTUALLY IMPORTANT . technology is a MEANS.
     the ranking changes; know it for right now.
     + THE OBJECTIVES THAT ARE ALWAYS TRUE: exist, pay
       people, keep a reputation. only stated when in danger.
THE CUSTOMER'S VIEW . measure success from where THEY
     stand. a user does not distinguish the parts of the
     chain: there are sites that work and sites that
     do not. holds for internal customers too.
PRIOR ART . many problems are not essentially new.
     the goal is to SOLVE the problem, not necessarily
     to write code to solve it.
```

Q Name the four risks. Which two are mirror images, and what single cause produces both?

Q Why is paying attention insufficient on its own?

Q What is the test for whether your current work is a waste of your time?

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Going deep in one domain	Depth, understanding, and the nuance that makes you worth having on hard problems.	All four risks. Your ranking distorts, other work looks easy, local defects vanish from view, and the purpose can detach.	Always, but with a deliberate correction running alongside — not instead of.
Deliberately clearing fog rather than waiting to absorb it	Context you would never have picked up passively, and a picture that is current rather than historical.	Time that produces nothing visible, spent on meetings and documents that are not your job.	Continuously. Parts of the map you cleared last quarter have fogged back up as priorities changed.
Building relationships with peers outside your group	An end to the echo chamber, honest criticism of your own group, and a more objective view of what everyone is doing.	Real time, and the discomfort of having firmly held beliefs made more nuanced.	When everyone you talk to agrees with you. That is a symptom, not a comfort.
Going beyond engineering — product, support, admin, legal	A whole new way of thinking about what is important to the business, and people who know things you cannot get elsewhere.	It looks least like your job of anything on this list.	Whenever your work affects them or theirs affects you — which is more often than it appears.

Checking for prior art before building	Better solutions, and less time reinventing wheels.	Slower start, and the deflation of discovering your special problem is ordinary.	Before creating some new thing. The unusual case is the genuinely novel problem, not the other way round.
Measuring success from the customer's point of view	The only measure that corresponds to whether anyone is helped, including failures in parts of the chain you do not own.	Metrics you cannot fully control, and responsibility that extends past your boundary.	Always. Availability numbers are useful and they do not tell the whole story, because they do not settle who defines “available”.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“I've been here two years, I know how things work.”	“I've been here two years, so there are things I've stopped seeing. What does the newest person on the team notice?”
“Our problems are pretty unique.”	“That's the local maximum talking. What's the prior art, inside and outside?”
“That other team's job is much easier than ours.”	“I've got a fish-eye lens on. Their domain is as nuanced as ours and I can't see it from here.”
“That config file has always been a mess.”	“How long has it been getting worse, and would I still call it minor if I saw it for the first time today?”
“This is the most important thing my org does.”	“It's important to us. Would it appear at all on a company-level map? And what does the company need right now?”
“We hit our availability target, so we're fine.”	“Who defined available? The user doesn't separate the parts of the chain — either it worked for them or it didn't.”
“Nobody told us the priorities changed.”	“That part of my map fogged up. I need a standing way to hear this, not a one-off correction.”
“Why are we still doing this project?”	“What goal was this serving, is that goal still live, and has it been solved another way in the meantime?”

PART 1 · QUESTION 1

An engineer has spent four years on a payments platform team and is now the team's technical authority. In one week: they argue that a proposed shared retry library cannot possibly work for payments because “our consistency requirements are unlike anything else in the company”; they describe the search team's ranking work as “basically a weekend of tuning”; a new joiner asks why the on-call runbook opens with a warning that the first three steps do not work, and the engineer answers that everyone knows to skip them; and, asked by a director what the platform work is ultimately for, they describe the architecture they are building rather than any outcome. The director suggests the engineer is “too close to it” and should care less.

(a) Name the risk on display in each of the four incidents, and give the mechanism the material attaches to each. **(b)** Two of these four are described as mirror images of each other. Say which, and what single cause produces both. **(c)** Assess the director's advice, and state what the actual nature of the problem is. **(d)** Give three concrete techniques the engineer could run, and for each, say which of the four risks it most directly attacks.

PART 1 · QUESTION 2

A tools team maintains an internal build system. Their dashboard reports 99.9% availability and they are proud of it, but the three teams that depend on them complain constantly; on investigation, most of the complaints are about a package mirror the tools team does not own, and about builds that succeed but take eleven minutes. The team lead proposes a company-wide programme to build a novel distributed cache, arguing that no existing system has their access pattern. Meanwhile the company has just announced a hiring freeze and a push to win back departing customers, and the team's quarterly plan does not mention either. The lead's own summary is: “we're doing fine, they just don't understand our constraints.”

(a) Explain, in the terms this part uses, why the availability figure and the complaints can both be accurate, and say what the team should be measuring instead. **(b)** Name what the novel-cache proposal is an instance of, and give the two checks that should precede it. **(c)** The quarterly plan ignores the freeze and the customer push. Name what the team has failed to establish, and separately name the category of company need that is only ever stated when it is in danger. **(d)** The lead is not lazy or unintelligent. Explain what is actually producing all three symptoms, and give the test for whether their work is a waste of their time.

2×2 — how far you can see, and how honestly you read it

COLUMNS: DO YOU KNOW WHAT THE BUSINESS CONSIDERS IMPORTANT RIGHT NOW? →

	Yes — including the objectives that are always true	**No — you know your own group's goals only**
You can see OUTSIDE your own group	**Perspective** You can place your work on a company-scale map, you know how your customers judge it, and you can tell whether now is the moment for foundations or for features. *Move:* this is the cell the rest of the booklet assumes. Hold it deliberately — it decays as priorities change and as you become an insider again. Ask a newer person what they see, and check that the ranking you are using is this quarter's.	**Well connected, badly aimed** You have peers everywhere and hear everything, and you still cannot say which of the things you heard matters, so you rank by whoever spoke most recently. *Move:* go get the business context — all-hands for your group and others, skip-level conversations, face time with customers and with the teams that depend on you. If you do not have context about why your work matters, ask for it.
You see your OWN GROUP only	**Loyal and mis-scaled** You can recite the company strategy and you still cannot judge your own work against it, because your only sample of what a hard problem looks like is your own. *Move:* the correction is other people, not more thinking. Build relationships with senior peers outside the group, listen to what other teams say about yours, and check prior art before deciding your problem is special.	**The silo** The local maximum feels like the summit, other teams' work looks trivial, the broken things nearby have gone invisible, and the reason for the project has quietly detached from the project. *Move:* all four risks are live at once, and the one that costs most is the fourth — a project running on after its goal died. Start by asking what this work was for and whether that is still true.

Read it as: the row is *reach* — how much of the map you can see — and the column is *calibration* — whether you can tell what is worth anything. Neither substitutes for the other, and they fail differently: reach without calibration produces busy people chasing whatever is loudest, and calibration without reach produces people who can recite the goal and still cannot locate themselves relative to it. Only the top-left lets you answer the question the whole part is really asking, which is whether the thing in front of you deserves the person in front of it.

CARRY-OUT RULE FROM THIS PART

Treat your own view as an instrument that drifts, and recalibrate it against people rather than against thinking harder. Depth and perspective are traded against each other, and no amount of care or good intent corrects the trade — only outside inputs do. The cheap standing habits: keep one relationship with a peer outside your group, ask the newest person what they see, and once a quarter say out loud what your work is for and check that the goal is still alive.

2

PART 2 OF 4

The terrain and the destination

How your organisation actually behaves, what is in your way, where decisions really get made — and where you are all supposed to be going.

TENTH-GRADER VERSION

- Knowing where you are isn't enough to get anywhere. You also need to know what the ground is like.
- Teams push against each other like slabs of rock. Where they meet you get cracks, ridges and arguments — and a new boss can rearrange the whole landscape overnight.
- Every company has a personality: how much it writes down, how secret it is, whether ideas come from the top or the bottom, how fast it changes, whether you get things done by filing a ticket or by asking a friend.
- There's a short list of things that block you: cracks between teams, gatekeepers, ground two teams both claim, fights everyone says are unwinnable, and official routes that nobody actually uses.
- Decisions get made somewhere. Find out where, and find out who the boss quietly checks with before agreeing to anything — it's often not the person you'd guess.
- And then: where is everyone going? If nobody knows, teams build for every possible future, or each picks its own and they don't fit together.
- You research physics because you want a railway. Forget the railway and you just keep researching things.

THE EVERYDAY VERSION

Two people are given the same instruction: get a shipment from the port to a town on the far side of the country. The first has a pin marking where they are and a pin marking the town, and sets off in a straight line. The second has a proper survey map. It shows that the direct line crosses a gorge with no bridge for sixty miles; that one valley is controlled by a toll keeper who lets nobody through without papers stamped in the capital; that two districts both claim the road along the ridge and neither will maintain it; that the desert to the south has beaten every party that has tried it, which is why the road stops where it does; and that there is an old drovers' track, on no official map, which everyone local uses because the new highway was built to the wrong places. Both know where they are. Only one can plan. And there is a third question neither pin answers, which the second traveller has also written on the map: **what is in the shipment for, and is the town still where it needs to go?** Because a perfectly executed delivery to a place nobody needs it is not a success, and you will only find that out if somebody asked.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Team tectonics	Domains of responsibility **smashing against each other** and forming an organisational terrain, complete with overlaps and conflict, ridges and chasms.	Reorganisations disrupt communication between groups that need to work closely together. Teams under heavy load entrench and put up barriers. **A new senior leader can cause an earthquake that reshapes the landscape overnight.**	vs “politics”, the word engineers use to dismiss this. Organisational skill is part of good engineering: considering the humans who are part of the system, being clear about the problem you are solving, understanding long-term consequences, and making trade-offs about priorities.

The three difficulties without a terrain map	What goes wrong when you set out without one.	**1.** Your good ideas do not get traction — being right is less than half the battle; you have to convince people you are right and, harder, **convince them to care**. **2.** You do not find out about the difficult parts until you get there — many obvious-seeming journeys have a crucial point nobody has got past. **3.** Everything takes longer — decisions that should be straightforward take quarters.	The third has a specific and unglamorous cause worth knowing: **planning cycles**. Announce an initiative immediately after the quarterly objectives have been set and you face a hard fight and may wait a quarter for any progress. Knowing where people were stymied before lets you take a different path, or **solve the hardest part first** so others believe it is worth their effort.
The seven cultural attributes	Sliders describing an organisation's personality. Most companies sit somewhere in the middle of each; it is hard to manoeuvre at either extreme.	**Secret or open** (is information currency, or is everything visible including messy drafts?) · **oral or written** (hallway decisions, or specification and sign-off for a one-line change?) · **top-down or bottom-up** (where do initiatives come from?) · **fast or deliberate change** · **back channels or front doors** (a direct message to a counterpart, or file a ticket and wait?) · **allocated or available** (is everyone's time already spent?) · **liquid or crystallized** (can you move in the structure, or do you wait your turn?).	vs the company's published values, which are **aspirational** — the real values are reflected in what actually happens every day. There is no right answer per slider; what matters is knowing where yours are, because the same behaviour is virtuous at one end and disqualifying at the other. Resharing something told in confidence loses your boss's trust in a secret culture; withholding information marks you as political in an open one.
Power, rules, or mission	A three-way typology of how organisations process information, from a mid-2000s paper by the sociologist **Westrum**.	**Pathological** — power-oriented, low cooperation, messengers shot, responsibilities shirked, bridging discouraged, failure leads to scapegoating, novelty crushed. **Bureaucratic** — rule-oriented, modest cooperation, messengers neglected, narrow responsibilities, bridging tolerated, failure leads to justice, novelty leads to problems. **Generative** — performance-oriented, high cooperation, messengers trained, risks shared, bridging encouraged, failure leads to inquiry, novelty implemented.	vs a judgement about whether people are nice. It is specifically about **information flow**, and the evidence attached is that high-trust cultures emphasising information flow have **better software delivery performance**. Its use is predictive: in a bureaucracy, plan ahead, stay within the rules and respect the chain of command; in a pathological organisation, take fewer risks and cover yourself when you do.
Chasms	Gaps opened by tectonics between organisations — classically between product-focused engineering and the infrastructure, platform or security teams serving them.	Group cultures, norms, goals and expectations evolve differently, making it hard to communicate, decide and resolve disputes. Smaller ones form *inside* an organisation too: **the edges of each team's defined responsibilities rarely line up perfectly**, and work and information fall into the gaps.	vs a communication problem you can fix with a meeting. The gap is structural, which is why the counter is structural too — see **bridging**, below, and defining your own scope so that it crosses the plates rather than sitting on one of them.

Fortresses	Teams or individuals who seem determined to stop anyone getting projects done.	You need their approval and cannot get time with them; or they decide your idea is bad before knowing what you are asking. To pass, you may need **a token of sponsorship from someone the gatekeeper respects**, or a password: proving you have mitigated every risk, completing lengthy checklists or capacity estimates, answering huge numbers of document comments acceptably.	vs obstruction for its own sake. **The majority are well intentioned — they gatekeep because they care**, trying to keep quality high and everyone safe. The two alternative routes both cost: a protracted battle can be so Pyrrhic you regret winning, and going the long way round complicates your journey **and loses you whatever wisdom the gatekeeper would have shared**.
Disputed territory	Work that more than one team believes it owns — or that is owned in pieces by several, none of whom can speak for the whole.	A system whose ownership is smeared across three teams, each responsible for a different aspect: asked whether migrating it is safe, each says “yes, as far as I know, but you should also ask…” and points at the next. Without an owner who can speak for the whole system there is no way to build confidence except to learn the whole thing yourself.	vs a simple ownership gap. The failure is not only slowness: when two teams must work closely and **lack the same clear view of where they are trying to get to**, projects fall into chaos, and the misalignment produces power struggles and wasted effort as both sides try to win the technical direction.
Uncrossable deserts	A battle other people consider unwinnable — a project too big, or a politically messy situation that always ends in a veto.	People have tried before, and any suggestion of trying again meets discouragement and ennui.	vs a reason not to try. The stated condition is evidential: you should have **enough evidence to convince yourself and others that this time will be different**. It is worth knowing going in that you may be picking an unwinnable fight.
Paved roads and goat tracks	A **paved road** is an official way that has been made the easiest way. A **goat track** is the undocumented route people actually use.	A self-service checklist that makes a new component safe to put into production is a paved road; know where those are and use them. The goat tracks are the undocumented search feature, the administrator who can set up an account, the one person who answers direct messages.	The trap is assuming the official route is the used route. **Sometimes the official way is the way everyone learns not to use**, because the new road does not lead to the places people actually want to go. Note the political catch: once you learn the goat tracks, teams may ask you not to document them, insisting people should use the new path and it will be better soon.
The room where it happens	The forum where decisions in your area are actually made — and the observation that **if you want to set technical direction, you need to be an insider in the group making the decisions**.	It might be a weekly managers' meeting that drifts into technical direction, a director's staff meeting, or a channel where an architecture group reaches consensus. If you cannot see it, ask someone you trust to walk you through where a particular decision came from — being clear you are not fighting the decision, only understanding the mechanism.	Three cautions. **There may be no room at all** — pronouncements made in one-to-ones with the most senior leader, or intended to be bottom-up and therefore often not made. **Some rooms you should not be in**, such as compensation and performance discussions if you are on the individual-contributor track. And **the room may contain less power than you think**; the people in it may be frustrated at not influencing the real decisions two levels up.

The cost of admitting you	What the people already in the room pay when you join, and which you have to offset.	**Adding someone to a group is rarely free.** Every extra person slows a meeting, extends discussions, and reduces attendees' willingness to be vulnerable or brutally honest; if the group is used to working together, **every new person resets the dynamic**.	So the case for admitting you must be about **impact to the organisation, not to you** — framing your exclusion as bad for your career will not move anyone. And once inside, as well as adding value you must **reduce the cost of including you**: show up prepared, speak concisely, be low-friction. Make the room worse at deciding and you will not be invited back.
The shadow org chart	The **unwritten structures through which power and influence flow** — probably not the same as the actual org chart.	Two named kinds of influencer: **connectors**, who know people right across the organisation, and **old-timers**, who wield influence regardless of rank or title simply from having been around a long time. People with rank and title trust them and rely on their judgement.	vs seniority. The practical signature is a director who is enthusiastic about your idea and then goes cold — because their first move in any decision is to check with someone several levels below them. **These are the people you need to convince before a change can happen**, and the influence lines are invisible when you join, so make friends early and ask how the place works.
Bridging	Making connections between parts of the organisation that would otherwise have **enormous information gaps**. It is one of the properties the typology uses to separate its three cultures.	The most impact often comes from going where everyone else fears to tread: sending the summary email nobody is sending, introducing two people who should have spoken a month ago, writing the document that shows how projects connect. The preemptive version: find someone whose work you do not understand, or whose work frustrates you, and go and talk to them.	vs relationship-building for its own sake. The claim underneath is causal — **the problems that slow down technology organisations are most often human ones**: teams that do not know how to talk to each other, decisions nobody feels empowered to make, and power struggles. It also has a structural form: define your scope so it crosses the plates and covers a whole system or problem domain, so you can catch dropped work and mediate conflicts.
The treasure map	The **destination and the stops along the way** — where you are all going and why you want to get there.	Uncovering it means taking a long view and evaluating the purpose of the work: is each project a goal in itself, or a milestone on the path to the actual goal? Sometimes you will discover there is no destination at all, or several incompatible ones.	vs a roadmap, which lists what is being built. This one has to **define the treasure** — give a clear definition of success — and it is shared, not held: tell the story of where you are, where you are going and why these steps, marking the hazards and shortcuts but not distractions. Tell the story of **where you came from** too, because it is surprisingly easy to feel you are getting nowhere.
The five costs of short-term-only	What happens to a group with no destination.	**1.** Harder to keep everyone going the same direction — either they go to the wrong place, or every decision becomes long and complicated. **2.** You will not finish big things, because bigger problems take multiple steps. **3.** You accumulate cruft. **4.** You get competing initiatives — several people rallying enthusiasm around completely different directions, all trying to do the right thing, producing chaos. **5.** Engineers stop growing.	The third has a precise two-branch mechanism worth memorising: without a destination a team either tries to **stay flexible enough to support every future state**, producing overcomplicated and hard-to-maintain solutions, or **makes local decisions** and risks becoming the weird edge case everyone else has to work around. The fifth is about muscle: a team used to short, clearly specified goals never builds the ability to find and fill the gaps between assigned tasks.

The technology tree	The analogy for why any given piece of work is being done: capabilities form a **directed graph**, and much of what you research is an unavoidable step toward something else.	In the strategy game the analogy comes from, you cannot build a railroad without bridge building and steam engines, and you cannot build steam engines without physics and engineering — so there is a point where you are researching physics **and your actual goal is a railroad**. You are not building the infrastructure layer for its own sake; you are building it so services start up quickly, so teams ship features faster. **The real goal is to reduce time to market.**	vs a dependency list. The point is directional: **unless you remember where you intended to go and keep working on it, you never get to ride the train**. Knowing the real goal is what lets you step back and evaluate whether a proposed piece of work gets you closer to it — which is a test you cannot apply to a project defined only by what it builds.

> **CONCEPT BOUNDARY — THE TERRAIN MAP IS A PREDICTION DEVICE, NOT A COMPLAINT**
>
> **What it sounds like:** a catalogue of the annoying things about your company — the gatekeeper, the useless official process, the reorg — which is to say, a licence to be cynical with better vocabulary.
> **What it actually is:** a set of facts each of which changes what you would do next. Knowing your organisation is deliberate rather than fast means an initiative needs a whole plan rather than an immediate increment. Knowing it is rule-oriented rather than mission-oriented means planning ahead and respecting the chain of command instead of building grassroots momentum. Knowing that the obstacle is a fortress rather than a chasm means finding a sponsor's token rather than sending another summary. Knowing the quarterly objectives were set last week means waiting three months is the fastest route.
> **Why that matters practically:** it converts the two most common failures of a technically correct person. The first is being right in the wrong month. The second is treating every obstacle as the same obstacle and applying the same remedy — usually a better document — to a chasm, a fortress, a disputed territory and a desert, which need respectively a bridge, a sponsor, an owner, and evidence that this time is different.
> **The trap:** mapping the terrain and stopping there, because the map is interesting and crossing the ground is not. The stated purpose of learning where the dragons are is **to go there**: the largest impact available is usually in the territory everyone else has agreed to avoid.

Anchor sketch — the ground you cross, and the place you are going

Legend: bold = a named feature or practice **reversed = the fact everything else answers** **Q = cover the page and answer this**

```
BEING RIGHT IS LESS THAN HALF THE BATTLE.
     you must convince people you are right AND
     convince them to CARE.
                       |
THREE DIFFICULTIES WITHOUT A TERRAIN MAP
     good ideas get no traction . you meet the hard part
     with no warning . everything takes longer
     -> the dull cause of the third: PLANNING CYCLES.
        announce just after objectives are set and you
        wait a quarter.
                       |
TERRAIN = TEAM TECTONICS. domains of responsibility
grind against each other. reorgs cut communication.
loaded teams entrench. a new leader is an EARTHQUAKE.
```

"politics" is the word engineers use to dismiss this. it is engineering: humans in the system, the real problem, long-term consequences, trade-offs.

SEVEN CULTURAL ATTRIBUTES . most orgs sit in the middle

SECRET <-> open **ORAL** <-> written

TOP-DOWN <-> bottom-up **FAST** <-> deliberate

BACK CHANNELS <-> front doors

ALLOCATED <-> available **LIQUID** <-> crystallized

published values are ASPIRATIONAL. the real values are what happens every day.

Q same act, opposite verdict: name one.

POWER, RULES, OR MISSION . how information flows

PATHOLOGICAL power . low cooperation . messengers SHOT . responsibilities shirked . bridging discouraged . failure -> SCAPEGOATING . novelty crushed

BUREAUCRATIC rules . modest cooperation . messengers NEGLECTED . narrow responsibilities . bridging tolerated . failure -> JUSTICE . novelty -> problems

GENERATIVE mission . high cooperation . messengers TRAINED . risks shared . bridging ENCOURAGED . failure -> INQUIRY . novelty implemented

-> high trust + information flow => better software delivery performance.

POINTS OF INTEREST . each has a DIFFERENT remedy

CHASMS gaps between orgs, and the unlined-up edges of team responsibilities. work falls in. -> BRIDGE.

FORTRESSES gatekeepers. MOST ARE WELL INTENTIONED -- they gatekeep because they care. -> a token of sponsorship, or the password. the long way round costs you their wisdom.

DISPUTED TERRITORY ownership smeared across teams. "yes as far as I know, but also ask..." -> find or make ONE owner who speaks for the whole.

UNCROSSABLE DESERTS fights others call unwinnable. -> not "never". EVIDENCE that this time differs.

PAVED ROADS / GOAT TRACKS official easy paths, and the undocumented ones people really use. sometimes the official way is the way everyone learns NOT to use.

WHERE DECISIONS HAPPEN

THE ROOM . to set direction you must be an INSIDER. beware: there may be no room; some rooms you should not be in; the room may hold less power than you think. admitting you is NOT free -- slower meeting, less candour, dynamic reset. so: argue ORGANISATIONAL impact, and REDUCE THE COST of including you.

THE SHADOW ORG CHART . unwritten influence. CONNECTORS (know everyone) and OLD-TIMERS (long tenure beats title). the director who goes cold checked with them.

KEEPING IT FRESH . announcement channels . walking the floor . lurking (calendars, agendas, newest channels) . reading time . checking in with leadership . talking with people

Q why does drifting from the tech cost twice?

```
   ------------------------------------------------------
   THE TREASURE MAP . where are we all going, and why
   FIVE COSTS OF SHORT-TERM ONLY
        1 hard to keep one direction  2 big things never finish
        3 CRUFT: support every future (overcomplicated) OR
          decide locally (become everyone's edge case)
        4 competing initiatives -- all well meant, chaos
        5 engineers stop growing (never fill the gaps)
   WITH a destination: no need for tight alignment on the way.
        each team routes itself. less hedging, less debt.
   THE TECH TREE . you research physics because you want a
        RAILROAD. the mesh is for fast startup is for shipping
        features: THE REAL GOAL IS TIME TO MARKET.
        -> forget the railroad and you just keep researching.
   SHARE IT . define the treasure = define success. mark the
        hazards, not the distractions. tell where you CAME
        FROM as well as where you are going.
```

Q Name the five points of interest and the distinct remedy each one takes.

Q Give the seven rows that separate a pathological, a bureaucratic and a generative organisation.

Q Give the two-branch mechanism by which a team with no destination accumulates cruft.

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Learning where your organisation's cultural sliders sit	Correct framing: an incremental path that shows value now, or a whole plan thought through, depending which you are in.	Nothing but the time — and the discomfort of admitting the published values are not the operating ones.	Before your first significant proposal, and again after any reorganisation or change of leadership.
Going through the fortress gate rather than round it	The gatekeeper's wisdom, and a route that stays open for everyone behind you.	Sponsorship you have to earn, checklists, capacity estimates, and comment threads answered to their satisfaction.	Almost always — but budget for it. The battle route can be so Pyrrhic that winning feels like losing.
Taking on an uncrossable desert	The thing nobody else will do, which is where the largest impact usually is.	Discouragement from everybody, and a real chance of confirming that it is uncrossable.	Only when you have evidence that this attempt differs from the failed ones — not enthusiasm, evidence.
Asking to join the room where decisions are made	Influence at the point of decision instead of reaction afterwards, and an end to being surprised.	A real cost to the people already there — slower meetings, less candour, a reset dynamic — which you must offset.	When you can argue it in terms of the organisation's goals and can show what you bring that is not already there.
Documenting the goat tracks	Newcomers stop losing weeks to the official route that does not work.	Teams may object: people *should* use the new path, and it will be better soon.	Judgement call. Weigh how soon “soon” really is against how many people are stuck today.

Defining your scope so it crosses the plates	You catch work that is being dropped, mediate conflicts, and can say with authority whether a proposed migration is a good idea.	You own the messiest parts of several systems, and belong entirely to none of the teams.	When work keeps falling into the gaps between teams and no single owner can speak for a whole system.
Insisting on a long-term destination	Teams that can route themselves, celebrate real milestones, and hedge less.	The argument itself, which may reveal several incompatible destinations or none.	When decisions keep reopening, initiatives compete, or you cannot say what the current project is a step toward.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“I don't do politics, I just do the engineering.”	“Considering the humans in the system *is* the engineering. Skipping it makes every change harder.”
“My proposal was obviously correct and it went nowhere.”	“Correct was less than half of it. Who needed to care, and did I ask in the right month?”
“That team blocks everything.”	“That's a fortress, and most fortresses gatekeep because they care. What's the password, and whose sponsorship opens the gate?”
“Nobody can tell me if this migration is safe.”	“Ownership is smeared across three teams. I need one owner who can speak for the whole system before I go further.”
“Everyone says that project is impossible.”	“It's a desert others have died in. What evidence do I have that this attempt is different?”
“There's an official process for this.”	“Is the paved road actually the road people use, or is everyone on a goat track?”
“The director agreed and then changed their mind.”	“Their first move is to check with someone. That person is on the shadow org chart and I haven't convinced them.”
“I keep being surprised by decisions.”	“Then I don't know where the room is. Let me ask someone to walk me through where the last one came from.”
“We're building the platform because we need a platform.”	“It's a node on the tech tree. The real goal is time to market — so does this step get us closer?”
“Let's stay flexible until we know the direction.”	“Supporting every possible future is one of the two ways cruft accumulates. The other is deciding locally. We need a destination.”

PART 2 · QUESTION 1

An engineer joins a large insurer and, six weeks in, writes an excellent proposal to consolidate four internal deployment tools into one. They post it in an open channel two weeks after the half-year plans were signed off. The head of platform is enthusiastic in the meeting, then stops replying; a colleague mentions that the head always checks with a quiet engineer who has been there eleven years. The security group, whose sign-off is needed, has not responded to three messages, and a peer warns that the last two people who proposed tooling consolidation were told it had been tried in the past and abandoned. There is also an official request process for platform work which, the engineer discovers, nobody uses — everyone messages a particular administrator directly. The engineer's response is to rewrite the proposal, adding forty pages of detail, and to post it again.

(a) Name each of the four obstacles in play, and give the specific remedy the material attaches to each. **(b)** Explain the enthusiasm-then-silence pattern using the correct term, and say what the engineer must do that a better document cannot do. **(c)** Two facts about *timing and channel* are working against the proposal. Identify both and say what each one predicts. **(d)** Assess the rewrite. State the general principle it violates, and give the order in which you would attack these obstacles instead.

PART 2 · QUESTION 2

A retailer's engineering organisation has no stated long-term technical direction. Over eighteen months: the checkout team has built an abstraction layer that supports three payment models “so we're ready for whatever we pick”; the fulfilment team has hard-coded its own event format and every other team now writes a converter for it; three separate engineers are independently campaigning for three different messaging platforms; a two-year data-consolidation effort was started and quietly abandoned at the eight-month mark; and the most junior engineers, asked to describe their work, can each describe their ticket queue but not what any of it is for. A director's diagnosis is that the teams lack discipline and need stricter delivery reviews. Separately, a staff engineer is asked to justify a service-startup project and answers that it will make service startup faster.

(a) Name each of the five costs of thinking only short-term, matching each to the evidence above. **(b)** The checkout and fulfilment teams have taken opposite approaches and both produced the same category of damage. Name that category, and give the two-branch mechanism. **(c)** Assess the director's diagnosis, and say what the organisation actually lacks and what having it would change. **(d)** The staff engineer's justification is incomplete. Name the analogy that explains why, and rewrite the justification so it can be evaluated.

2×2 — how the information moves, and whether anyone named the destination

COLUMNS: IS THERE ONE AGREED DESTINATION? →

	Yes — a shared, stated definition of success	No — none, or several incompatible ones
Information FLOWS (bridging encouraged)	**Navigable** People carry news across boundaries, failure produces inquiry rather than blame, and every team can route itself because everyone knows where the destination is. *Move:* this is where big things get finished, and it is the cell to protect rather than admire. Keep the terrain map fresh — announcement channels, walking the floor, reading time, and talking to people — because a reorganisation or a new leader can move you out of it overnight.	**Energetic drift** Communication is excellent and produces competing initiatives: several people rallying enthusiasm around completely different directions, each doing the right thing, adding up to chaos. *Move:* the missing artefact is a destination, not more communication. Write up where you see confusion or misalignment and share it, which forces the disagreement into the open — and if it turns out nobody ever chose, the job becomes choosing.
Information is HOARDED or stuck in channels	**A destination nobody can reach** The goal is stated and agreed, and the work still stalls: messengers are neglected or shot, teams entrench, and the gaps between them swallow whatever falls in. *Move:* bridging is the highest-value work available — the summary nobody is sending, the introduction nobody has made, the document showing how the projects connect. Consider defining your scope so it crosses the plates.	**Fog** Nobody knows where anyone is going and nobody would hear it if they did. Every team builds for every future or for none, and each obstacle is met for the first time by everyone who meets it. *Move:* do not attempt both at once. Clear enough terrain to find out who actually decides things, then use that to get a destination named — in this cell the destination cannot be announced into existence.

Read it as: the row is whether the organisation can *carry* information and the column is whether there is anything worth carrying. They fail in ways that look alike from inside and need opposite treatments: a place with flow and no destination feels busy and productive right up to the point where nothing large has shipped in a year, and a place with a destination and no flow feels aligned in every meeting and loses the work in the gaps between the teams that attended.

CARRY-OUT RULE FROM THIS PART

Before you argue that something should happen, find out how things happen here — and before you plan a route, find out whether anyone has named the place you are going. Each obstacle has its own remedy and they are not interchangeable: bridge a chasm, find sponsorship for a fortress, create a single owner for disputed territory, and bring evidence to a desert. And keep the map current, because the facts you rely on are the ones most likely to have quietly stopped being true.

3

PART 3 OF 4

Making a huge project tractable

Why big projects are hard, the context you have to go and get, the structures that hold it, and what driving actually means.

TENTH-GRADER VERSION

- Big projects are rarely hard because the technology is hard. They're hard because nothing is clear: what it's for, who decides, and what the old system does.
- Feeling overwhelmed at the start is normal and it's not about you. It's just missing information — and there's a list of what to go and find.
- Nobody on the project works for you. You're responsible anyway, including for the bits that aren't anybody's job.
- Write down who does what before you need to, not after two people discover they were both doing it.
- Explore before you design. If your goal is just a description of what you're building, you haven't explored.
- Write things down. Being wrong is fine — people correct you. Being vague is worse, because nobody can.
- Tell people what your progress means, not what you did. And don't say it's fine when it isn't; green on the outside and red inside gets found out at the end.
- Something will go wrong. Plan as if you know that, because you do.

THE EVERYDAY VERSION

You are asked to organise a large public event. Nobody who will work on it reports to you. There is a budget of unclear size, a date somebody announced in a newspaper before asking you, four different people who each believe they are in charge of the programme, a venue with a history nobody wants to discuss, and a list of “requirements” that turns out to be three emails you were not copied on. The temptation is to start doing something visible — book a band, print a poster — because doing feels better than not knowing. The person who succeeds does the opposite: they spend the first stretch finding out why the event is happening at all, who is paying, what would count as it having gone well, which dates genuinely cannot move, what happened the last time somebody tried this, and who is actually deciding the programme. Then they write it all down in one place, agree who owns what before anybody has to be told, and set a first small milestone that real people can attend, because **a small thing that happens teaches you more than a large thing that is still being planned**. And they assume in advance that something will go wrong — not out of pessimism, but because it makes the disruption interesting rather than devastating when it arrives.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH

What actually makes a project hard	Usually **not that you are pushing the boundaries of technology** — it is that you are dealing with **ambiguity**: unclear direction; messy, complicated humans; or legacy systems whose behaviour you cannot predict.	What makes a great project lead is rarely genius: it is perseverance, courage, and a willingness to talk to other people.	vs technical difficulty, which is what most people prepare for. The consequence is that the lead's job is mostly information work, and that **you are responsible for the whole problem, including the parts in the fissures between teams and the parts that are not really anyone's job** — while nobody on the project reports to you.
The overwhelm, and the five things that reduce it	The normal state at the start of a big project. It takes time and energy to build the mental maps that let you navigate, and at the start it can feel like more than you can handle.	**1.** Create an anchor — one document, just for you, an external part of your brain: uncertainty, rumours, leads, reminders and to-dos, so you can return to what Past You thought mattered. **2.** Talk to your sponsor, arriving with a written description of what you think they want and what success looks like, and ask whether they agree. **3.** Decide who gets your uncertainty. **4.** Give yourself a win. **5.** Use your strengths.	vs a signal that you are not up to it. "**That feeling of discomfort is called learning**", and **the difficulty is the point — if it were not messy and difficult they would not need you**. It may still signal a depleted resource: check whether you are out of time, out of energy, or short of a specific skill. Two details matter. Your junior engineers are **not** the right people to receive your fear — they look to you for safety — so find one peer or mentor who will say "yes, this is hard for me too". And **the start of a project is when it is easiest not to know things**; do not waste that period.
The eight points of context	What you have to clarify for yourself and everyone else before the project can be said to have begun.	**Goals** (why this one, out of everything?) · **customer needs** · **success metrics** · **sponsors, stakeholders and customers** · **fixed constraints** · **risks** · **history** · **team**.	vs a project brief, which is a document someone hands you. This is an *investigation*, with two uncomfortable results: understanding the "why" **might make you reject the premise of the project**, since completing something that will not achieve the goal wastes everyone's time; and describing the constraints honestly stops you **being mad at reality for not being as you wish it to be**.
Success metrics	An agreed, ideally measurable statement of what it will mean for this to have worked — agreed with your sponsor and the other leads.	Software projects sometimes implicitly measure progress by how much code is written, but **the existence of code tells you nothing about whether any problem has actually been solved**; sometimes the real success is *deleting* code. Metrics can be humble and still work: for one migration, time spent keeping the system healthy, and the number of complaints about things not working as expected.	vs a delivery plan. And there is an asymmetric rule attached: **if you initiated the project, be even more disciplined about defining success**. Credibility and social capital can carry a project for a while on other people's belief in you, but you cannot be certain you are right, and that capital can go down as well as up — so treat your own ideas with the most scepticism and get measurable goals in place quickly.

Defining roles	Laying out each leader's responsibilities at the **start** of the project, rather than when two people discover they are doing the same job or work is slipping through the cracks.	The simplest version is a table of leadership responsibilities against who owns each. The more formal tool is **RACI**: **R**esponsible (doing the work), **A**ccountable (delivering it and signing it off — **only one person per task**), **C**onsulted (asked their opinion), **I**nformed (kept up to date).	The value is not the matrix but that it is an **uncontroversial structure**, giving you a way to broach the conversation without it being weird. It breaks two named patterns: **never making a decision because nobody knows who the decider is**, so discussion runs forever, and **relitigating every decision** for want of a process. And as lead you **implicitly fill any role nobody is in** — growth, requirements, project management — which adds up.
Agreeing on scope	Deciding what you will actually try to do, using the **project management triangle** — time, budget and scope in tension. The blunt form is “fast, cheap, good: pick two”.	It feels obvious and is somehow easy to forget: **if you have fewer people, you cannot do as much**. So set a first milestone, put a date beside it, and describe what it looks like — what features are in, what a user can do.	vs a schedule. The reframing that does the work is **milestones as beta tests**: every milestone should be usable or demonstrable, giving users and stakeholders another chance to give feedback. That has a price — **each increment can change the requirements for the next**, because changing what users can do helps them realise what else they want — and a benefit: they may tell you early that you are on the wrong track.
Estimating time	Almost nobody is good at it, plausibly because every project differs and the only way to know how long something takes is to have done exactly that thing before.	The two most common pieces of advice are to break work into the smallest tasks you can, and to assume you are wrong and multiply by three. The preferred alternative: **often the only way to determine the timetable for a project is by gaining experience on that same project** — so deliver small slices, learn how long your team takes, and update the schedule each time. Practise estimating even when it does not matter, and keep a log of how it goes.	vs your own team's capacity, which is only part of it. Estimation must include the teams you depend on: **the later you tell another team you need something, the less likely you are to get it**, and if they do scramble, you are interrupting their previous work and **disrupting their estimates for their other projects**.
Exploring	The stage before design, in which you work out the project's true needs and evaluate possible approaches — and you do it **before** deciding how to build anything.	Be suspicious of a brand-new project that already has a plan full of implementation details; on day one you do not have enough information for granular decisions. Different teams have different mental models of what you are achieving; some may have agreed to help **because they think your project will also achieve some other goal they care about — and they might be wrong**; some use the same words for different things.	The stated test for whether you have explored enough is precise: **if it is difficult to articulate the goals — or the goals are just a description of your implementation — you have not spent enough time here**. The target is to be able to explain concisely what each team wants **in a way they will agree is accurate**, and to have both an elevator pitch and a clear description of what you are *not* doing.

Clarifying	Reducing the complexity of a big messy project by building shared understanding — giving everyone a mental model of what you are all doing.	Three tools. **Mental models**: hook the new concept to something the listener already knows, by analogy or example; it does not have to be perfect, only close enough to make a chain back to something understood. **Naming**: build a shared vocabulary, because even common words like *user*, *customer* and *account* can mean different things in finance, marketing and engineering — use their words, or supply a glossary. **Pictures**: before-and-after images, nesting for containment, parallel shapes for parallel ideas, a ladder or tree or pyramid for hierarchy.	vs communicating more. It is your job for an incentive reason: **you have cause to spend time understanding the tricky concepts; the people you ask for help have a different focus and may not try as hard**, so unless you reduce the complexity they may optimise for the wrong outcome or muddy your story. On pictures, mind existing associations: a cylinder reads as a datastore, and colour reads as approval or prohibition whether you meant it or not.
Designing in writing	Writing the plan down, because it is very difficult for many people to achieve something together without shared understanding — and **you will not know whether they understand and agree until you write it down**.	Absence of objection is not agreement: colleagues may not have internalised the plan, and **their implicit agreement may not mean anything**. It need not be long — a short, snappy read can be perfect. **A written design is a very cheap iteration.** Minimum headings: **context** (title, author, at least one date, and status), **goals**, **design**, **security, privacy and compliance**, and **alternatives considered / prior art**. Often-useful additions: background, trade-offs, risks, dependencies, operations.	vs an architecture review. Review here explicitly includes **whether you are solving the right problem at all** and whether your assumptions about other teams are correct — an obvious path for one team may break another org's workflows. Two headings keep you honest: the goal **must not contain implementation details**, since a goal that has chosen the technology has made a major design decision without justifying it; and omitting **alternatives considered** because you considered none signals you have not thought the problem through. A section you think irrelevant — security, say — should still say **why**.
Wrong is better than vague	It is a better use of your time to be wrong or controversial than to be hand-wavy — **if you are wrong, people will tell you and you will learn something**, and you can change direction.	Two precision techniques. **Be clear who or what is doing the action for every verb**: passive constructions like “the data will be encrypted in transit” obscure information and make the reader guess, so write “the client will encrypt the data before it is transmitted”. And **add a noun after “this” or “that”**, even if you just mentioned the thing — “to solve this shortage” rather than “to solve this”.	vs recklessness. The point is informational: vagueness is chosen to avoid argument, and it works — nobody can argue with it, so nobody can correct it. Disagreement about your design does not mean you must change course; **it gives you information you would not otherwise have had**.

The design pitfalls	Recurring failures to catch in your own documents before a reviewer has to.	**It's a brand-new problem** (it almost certainly is not) · **this looks easy** · **building for the present** · **building for the distant future** · **every user just needs to…** · **we'll figure out the difficult part later** · **solving the small problem by making the big problem harder** · **it's not really a rewrite** (but it is) · **but is it operable?** · **discussing the smallest decisions the most**.	Each has a specific tell. “This looks easy” is the failure to internalise that other domains are as rich and complex as your own — **if it seems trivial, it is because you do not understand it**. “Distant future” is answered by the rule that **“we might need it later” is not a good enough justification**. “Every user just needs to” fails at scale: five users can be taught the arcane rules, hundreds will get it wrong, and **anything requiring humans to change their behaviour is part of the design**. Operability has a test — **if you struggle to remember how it works at 3 p.m., you will not understand it at 3 a.m.** — and a staffing rule: name who is on call in the document, and if it is your own team, have more than three people and ideally at least six. The last is **bikeshedding**, from the 1957 law of triviality: a trivial issue is easier to discuss than a difficult one, so that is where teams spend their time.
Sharing status	Making it easy for stakeholders, sponsors and waiting teams to find out what is going on — **explained in terms of impact** rather than activity.	They probably do not care that you stood up three services; they care what users can do now and when they will be able to do the next thing. **Too much detail can obscure the overall message** and make it harder for the audience to take away the conclusion you intended. If they really do want everything, lead with the headlines and spell out the takeaways — practise following facts with “that means…”.	The dishonest version has a name: the **watermelon project** — green outside, red inside. Reporting green while hoping to sort it out risks an unpleasant surprise at the end. Note the symmetric error: a delay that **does not change your delivery date** may not be relevant, and raising it can look like escalating something that needs no escalation.
Navigating	Handling the thing that goes wrong — because **it is inevitable that you will meet roadblocks**; you simply do not yet know which.	A core technology turns out not to fit, the business changes direction, someone vital quits. Going in assuming something will go wrong makes you more flexible, so the roadblock is interesting rather than infuriating. As the person at the wheel you are accountable: you do not get to say the project is blocked and there is nothing to be done — **you reroute, escalate to someone who can help, or break the news that the goal is now unreachable**.	vs replanning. Two further rules. The bigger the change, the more likely you must **go back to the start, build context again, and treat the project as restarting**. And in communicating it, do not create panic or let rumours start: give people facts and **be clear about what you want them to do with the information**. Asking for help is not failure — your manager's job is to make you successful, and if you do not say you are stuck it is harder for them to do it.

> **CONCEPT BOUNDARY — DRIVING IS AN ACTIVITY, NOT A POSITION**
>
> **What it sounds like:** being the project lead — the person with the title, in the meetings, whose name is on the plan. On that reading, driving is something you *are*.
> **What it actually is:** a claim borrowed from the literal act. Driving does not mean putting your foot on the accelerator and going straight; it is active, deliberate and mindful — choosing your route, making decisions, and reacting to hazards on the road ahead. Every technique in this part is one of those three: exploring and gathering context is choosing the route, roles and written designs are making decisions in a way that sticks, and navigating is reacting to hazards.
> **Why that matters practically:** it explains what the passive version looks like, which is not laziness but plausible activity. The passive lead attends every meeting, answers every question, writes a lot of code, reports status weekly — and never makes anyone say what success means, never writes the design down because everyone seems to agree, never asks why the project exists, and finds out about the crucial obstacle when the project reaches it. Nothing about that looks like neglect from outside.
> **The trap:** substituting visible work for the driving. Contributing code is the classic form, and it has real benefits — depth of understanding, spotting problems, feeling the cost of your own architectural decisions. But watch whether you are taking on work you know how to do, with a short feedback loop, **in place of** the big design decisions and the organisational manoeuvring. And if you do take on the biggest problems, remember your time is less predictable than everyone else's, which is exactly how a lead becomes the bottleneck.

Anchor sketch — from overwhelm to a project that is being driven

Legend: bold = a named stage or tool **reversed = the fact everything else answers** **Q = cover the page and answer this**

THE DIFFICULTY IS AMBIGUITY, NOT TECHNOLOGY.
unclear direction . messy humans . legacy systems
you cannot predict.
it is rarely genius: perseverance, courage, and a
willingness to talk to other people.
nobody reports to you. you own the result ANYWAY --
including the fissures between teams and the work
that is nobody's job.
|
OVERWHELM is normal. "that feeling of discomfort is
called learning." if it were easy they would not
need you. (but check: time? energy? a missing skill?)
FIVE THINGS anchor doc (external brain) . talk to
the SPONSOR with a written guess at what they want .
decide WHO GETS YOUR UNCERTAINTY (not the juniors --
they look to you for safety) . give yourself a win .
use your strengths
-> the start is when it is EASIEST NOT TO KNOW THINGS.
|
BUILD CONTEXT . eight points
goals (may make you REJECT the premise) . customer
needs . success metrics . sponsors+stakeholders .
fixed constraints (stop being mad at reality) .
risks . history (respect what came before) . team
Q why does "how much code is written" fail as a metric?
Q who must be MOST sceptical about a project's value?
|
GIVE IT STRUCTURE
ROLES . decide at the START. RACI: Responsible,

```
            Accountable (ONE per task), Consulted, Informed.
            the superpower is an UNCONTROVERSIAL structure.
            kills: no decision (no known decider) and
            relitigating everything.
            you implicitly fill every empty role.
       SCOPE . the triangle: time / budget / scope.
            fewer people => less done.
            MILESTONES AS BETA TESTS: each one usable or
            demonstrable. price: each increment can change
            the NEXT one's requirements.
       TIME . the only way to time a project is experience
            ON THAT PROJECT. deliver slices, re-estimate.
            tell dependent teams EARLY: late asks get less,
            and a scramble wrecks their other estimates.
       LOGISTICS . meetings . informal channels . status .
            ONE documentation home . dev practices . kickoff
                        |
  DRIVE IT . not foot down and straight ahead: choose the
       route, make decisions, react to hazards.
  EXPLORE  before you design. suspect any day-one plan full
       of implementation detail. teams hold different mental
       models; some agreed to help for a DIFFERENT goal.
       -> TEST: if the goals are hard to state, or are just a
          description of your implementation, keep exploring.
       -> produce an elevator pitch AND what you are NOT doing.
  CLARIFY  models (hook it to what they know) . naming (user,
       customer, account differ by department) . pictures
       (beware: a cylinder reads as a datastore)
       -> you have an incentive to understand the hard parts.
          the people you ask for help do not.
  DESIGN   write it down; implicit agreement means nothing.
       A WRITTEN DESIGN IS A VERY CHEAP ITERATION.
       MINIMUM: context . goals . design . security/privacy/
       compliance . alternatives considered
       -> the GOAL must not contain implementation details.
       -> no alternatives section = you did not think it through.
       WRONG IS BETTER THAN VAGUE. name the actor for every
       verb; put a noun after "this".
  PITFALLS  brand-new problem . this looks easy (if it seems
       trivial you do not understand it) . building for the
       present . for the distant future ("we might need it
       later" is not a justification) . every user just needs
       to . difficult part later . small fix that hardens the
       big problem . not really a rewrite . IS IT OPERABLE
       (3 p.m. vs 3 a.m.; >3 on call, ideally 6+) .
       BIKESHEDDING (1957 law of triviality)
  COMMUNICATE  status in IMPACT, not activity. lead with
       headlines. "that means..."  no WATERMELONS: green
       outside, red inside.
  NAVIGATE  something WILL go wrong. you reroute, escalate,
       or say the goal is unreachable. big change => restart
       the context work. give facts, not panic.
```

Q Name the eight points of context, and which one can make you reject the project.

Q Give the test for whether you have explored enough, and the five minimum design headings.

Q What are the two failure patterns that defining roles exists to break?

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Exploring before designing	A problem framing everyone will agree is accurate, and a chance to discover the obvious solution does not solve the real problem.	Time at the start with nothing visible to show, and the discomfort of abandoning the architecture you arrived with.	Unless the problem is genuinely straightforward — in which case ask whether it needs someone at your level at all.
Writing the design down	You find out whether people actually agree; hazards get pointed out early; a very cheap iteration.	Writing time, and exposure — you have to commit to something specific enough to be wrong.	Whenever multiple people need the same understanding. Length should follow the culture, not the ambition.
Milestones that are usable or demonstrable	Real feedback, an early warning that you are on the wrong track, and the motivation of a reachable goal.	Each increment can change the requirements for the next, so the plan will not hold still.	Almost always, and especially when the biggest risk is building something nobody ends up using.
Formal role definition such as RACI	An uncontroversial way to raise who decides, ending both endless discussion and endless relitigation.	Overkill for small efforts, and process for its own sake if nobody was confused.	When there are several leaders, or when you notice decisions are being reopened rather than made.
Contributing code as the lead	Depth of understanding, earlier problem-spotting, engagement, and feeling the cost of your own architectural decisions.	Your time is less predictable than anyone's, so work on the critical path makes you the bottleneck — and it can crowd out the harder, more important decisions.	When the work is not time-sensitive, and ideally when it is a lever: a pattern others will copy, rather than a task you took from someone.
Reporting a status that is not green	Help while there is still time to use it, and a reputation that survives the end of the project.	An uncomfortable conversation now, in exchange for a much worse one later.	As soon as a key milestone is genuinely at risk. Not for a delay that does not move your delivery date.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“This project is too big, I don't know where to start.”	“This is an information problem. I need the eight points of context, starting with why we're doing it at all.”
“I'm not sure I'm the right person for this.”	“The difficulty is the point — if it were tidy they wouldn't need me. Am I actually short of time, energy, or a skill?”
“We'll sort out who does what as we go.”	“Let's agree the responsibilities now. Otherwise we'll either never decide anything or decide it repeatedly.”
“Our goal is to build an event-driven pipeline.”	“That's an implementation, not a goal. If I can't state the goal without naming the technology, I haven't explored enough.”

“Nobody objected, so we're aligned.”	“Silence isn't agreement. I'll write it down — a written design is a very cheap iteration.”
“I'd rather keep the design flexible for now.”	“Vague can't be corrected. I'd rather be wrong and told than unarguable.”
“The users just need to remember not to do that.”	“Five users can be taught; hundreds will get it wrong. Behaviour change is part of the design.”
“We've spent the whole review arguing about the naming.”	“Classic bikeshedding — the trivial item is the easiest to discuss, so it eats the time meant for the hard one.”
“It's basically fine, we'll catch up.”	“That's a watermelon. If we can't hit the milestone, the status isn't green and someone needs to know now.”

PART 3 · QUESTION 1

An engineer is named technical lead of an eighteen-month effort to replace a logistics scheduling system used by four teams. On day three they are handed a document titled “Goals”, whose first line reads “build a streaming scheduler on a distributed queue”. Two of the four teams have agreed to help; one of them privately expects the project to also retire a reporting tool the lead has never heard of. There is no product manager, and nobody has said who signs off on user-visible behaviour; a director and an engineering manager have each described themselves as “driving” it. The lead's plan is to spend the first month writing the core scheduling engine themselves, since it is the hardest part and they know it best, and to report progress as a percentage of the engine completed.

(a) Apply the stated test to the “Goals” document and say what it reveals; name the stage that has been skipped and what it is supposed to produce. **(b)** The second team's private expectation is a specific named hazard. Name it and say what the lead should do about it. **(c)** Name the structure missing around leadership, give its four roles with the constraint attached to one of them, and give the two failure patterns it exists to prevent. **(d)** Assess the first-month plan on two separate grounds — what it does to the lead's availability, and what the proposed progress measure fails to tell anyone.

PART 3 · QUESTION 2

A design document for a new internal permissions service is circulated. Its design section says “access will be validated at request time and the token will be refreshed as needed”; it has no alternatives section, because the author says no alternatives were considered; the security section reads “not applicable”; and it argues for automatic multi-region failover on the grounds that “we might need it later”. It notes that migrating existing callers “just needs each team to add one header” — there are roughly two hundred caller teams — and that the trickiest case, a legacy batch importer, will be handled “in a later phase”. The review meeting spends fifty of its sixty minutes debating the service's name. The author is a strong engineer who deliberately kept the document loose to avoid arguments.

(a) Name the four design pitfalls present, and for each give the reasoning the material attaches to it. **(b)** Rewrite the quoted design sentence using both stated precision techniques, and name each technique. **(c)** The author's reason for keeping it loose sounds prudent. Give the principle that answers it and explain the mechanism by which vagueness costs more than being wrong. **(d)** Name what happened in the review meeting, give its origin, and say what two sections would have been required even if the author believed they had nothing to say.

2×2 — do you know why, and is anyone steering

COLUMNS: IS THE PROJECT BEING ACTIVELY DRIVEN? →

	Yes — route chosen, decisions made, hazards met early	**No — plausible activity, nobody choosing**
The GOAL and success metric are agreed	**Tractable** Everyone can say why the project exists and what would count as it having worked; milestones are demonstrable. *Move:* keep the destination visible, re-estimate from delivered slices, and report status in impact.	**A good idea, drifting** The goal is sound and nobody is choosing the route: decisions stay open and the obstacle is met on arrival. *Move:* name the decider, write the design down, and set a milestone near enough that reality arrives in pieces.
The goal is UNSTATED or is really an implementation	**Efficient in the wrong direction** Strong execution on a project whose goals describe its architecture. It hits its dates; nobody can say what it solved. *Move:* go back to exploring: state the goal without naming a technology, and check that your helpers share your reasons.	**Overwhelm, institutionalised** No agreed purpose, no owner of the result, and nobody went to get the context that would resolve it. *Move:* treat it as day one — an anchor document and a sponsor conversation. Rejecting the premise is legitimate.

Read it as: the row is *direction* and the column is *steering*. Both are here because the right-hand cells look healthy from inside: one has meetings and status updates, the other hits its dates, and neither can say what would make it worth doing.

CARRY-OUT RULE FROM THIS PART

Before you build anything, be able to state the goal without naming a technology — and before you finish, be able to state what would count as success without counting your own output. Context gives you the first, metrics the second; driving is checking that both are still true when the ground moves.

4

PART 4 OF 4

The influence you did not ask for

Why your behaviour becomes other people's standard, and the four attributes you are modelling whether you mean to or not.

TENTH-GRADER VERSION

- Once people think you know what you're doing, they stop checking your work and start copying it.
- That means what you actually do becomes the rule, no matter what the rules say. If the senior people approve code reviews without reading them, everyone will.
- Four things are worth being: good at the work, the responsible adult, someone who remembers why any of this is happening, and someone who thinks past this month.
- Saying “I don't know” out loud is one of the most useful things a senior person can do, because it makes it safe for everyone else.
- When something goes wrong, say what happened and fix it. Don't hide, don't blame, and don't let anyone else take it.
- Some work keeps a team running and isn't on anybody's job description. Do that work, and let the junior people do the technical work instead.
- Stay calm. Big problems should get smaller when you touch them, not bigger.
- Someone will have to turn your system off one day. Build it so that's possible.

THE EVERYDAY VERSION

A new cook joins a kitchen. Nobody hands them the standards; they watch. If the head chef tastes everything before it goes out, the new cook tastes everything. If the head chef wipes down as they go, so does everyone. And if the head chef, in a rush one evening, sends a plate they know is not right, then the kitchen has just learned that on a busy night the rule is suspended — a thing that was never said aloud, never written down, and is now the standard. The awkward part is that this happens whether or not the head chef is trying to teach anybody anything. It is not a lesson; it is a leak. Two more things follow. First, the cook who admits they have never plated this dish and asks to be shown makes it safe for the next person to ask, which is how a kitchen where people learn quickly gets built. Second, when something is dropped and broken, everyone watches what happens next much more closely than they watch the cooking — because **what happens after a mistake is the clearest statement anyone in the room will ever hear about how this place works**.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH

Passive influence	The effect you have **just by the way you act** as an engineer and as a person — as distinct from deliberately setting out to change your organisation.	At this level, people assume you know what you are talking about. **Your work will be a little less checked and your ideas considered more credible**, and rather than guiding you, people will look to you for guidance. Hence the warning not to think out loud: you can find out a month later that people are discussing your half-baked idea as though it were already a project.	vs mentoring, sponsorship or advocacy, which are things you choose to do. This one is not opt-in. **How you behave is how others will behave**, and the consequence runs in both directions — being mature, constructive and accountable tells new engineers that is what a senior engineer is, and so does being condescending, impossible to please, or never available.
Values are what you do	Written values and engineering principles describe the aspiration; **the clearest indicator of what a company values is what gets people promoted**.	A stated commitment to collaboration is undermined if people reach senior levels through "heroic" solitary efforts. Principles describing thorough code review are undone if senior engineers approve changes without reading them — **everyone else will rubber-stamp too**.	vs hypocrisy at the top. The mechanism is imitation rather than dishonesty: the work you do is implicitly the type and standard of work that others will see as correct and emulate. Note also that this extends past the technical — **engineering is also how you talk to humans**, so being a good engineer sometimes reduces to being a good colleague.
Attribute 1 — being competent	Taking on things that need doing and **reliably doing them well**. It is built from knowledge and skills, self-awareness, and high standards.	Experience comes through **time, exposure and study — not innate aptitude**; nobody is born a skilled engineer any more than a skilled weaver. One professional body's grades require 8+, 10+, 15+ and 20+ years for successive levels; software is less rigorous, but staff engineers typically have at least 10 years and principal engineers at least 15.	vs knowing the most about every topic — sometimes you enter a new domain knowing the least, and that is fine. Two consequences. **Do not rush past your prime learning years**: the advice quoted is never to accept a managerial role until you are solidly senior, meaning at least seven years writing and shipping code and definitely no fewer than five — and the same reasoning applies to senior individual-contributor roles. And **be wary of drifting so far from the technology that you are only learning how your company operates**.
"Not technical enough"	A framing worth pushing back on, because it **claims to describe who someone is instead of the skills they have** and leaves no actionable path.	Be more precise. Do you mean they have not yet spent enough years doing hands-on technical work? That they do not yet know the domain you are working in? That they are missing specific skills — and which ones?	vs an honest assessment. The replacement is not politeness; it is that each precise version has a remedy and the vague one does not. It also connects to how expertise transfers: entering a new domain, **your general experience should help you recognise the shapes of problems** even when the specifics are unfamiliar — the broader your experience, the more hooks you have to hang new knowledge on.

Self-awareness — both directions	Being **confident and honest with yourself about what you know and what you do not**. Competence means having well-founded confidence that you can solve the problem — it does not require arrogance, jargon or showing off.	Admitting what you know: **do not set your bar at "best in the industry"**, because you can always think of someone better and holding back out of modesty helps nobody. Admitting what you do not: bluffing costs you the chance to learn, may produce bad decisions, and wastes the example — asking for the explanation from first principles is a shortcut worth using.	vs humility as a virtue. The argument is throughput: **we spend a huge amount of our working lives trying to get a shared model of the world into our brains** so we can collaborate and decide, and it slows everything down when people bluff or quietly disengage rather than be caught not knowing. **If senior people can admit they do not know things, everyone else will too.** A third piece: your context is not the universal context — and if you can explain a topic in plain language so a nonexpert can hook it onto something they already understand, **that is a real indicator of expertise**.
High standards, and owning mistakes	Knowing what high-quality work looks like and aiming for it **in everything you do, not just the parts you enjoy most** — and responding well when you get it wrong.	Write the clearest documentation you can; be the first person to know if your software breaks. Sometimes the right move is duct tape as fast as possible — but **make that determination based on the problem you are solving, not on how interesting the work is to you**. Seek out criticism: ask for review, invite people to poke holes in an idea you love.	The mistake half is where most of the modelling happens. **Do not beat yourself up, and do not deny the impact or insist it was not really your fault.** Admit what happened, then set out to fix it; communicate quickly; and **if there is any risk someone else will be blamed, clarify that they did nothing wrong**. A retrospective helps, and being matter-of-fact about your own part is what makes it possible for others to be. **A leader open about their mistakes is a big boost to the team's psychological safety.** One boundary on criticism: your solutions are not you, and criticism of your work is not criticism of you.
Being reliable	Being the person others do not need to double-check. The compliment offered as the definition: "**Alex is going to be in that meeting, so I don't need to go.**"	That sentence is not only about information being represented; it means **the right thing will happen and the situation will be managed**. Reliability builds up the way credibility does — as people watch you do the work and get things under control.	Part of it is **finishing what you start**: sticking with it after it becomes boring or difficult. And the honest exception is explicit — if you stop deliberately because the project is not the right use of resources, **own and communicate that decision**. Quietly trailing off is the failure; a stated, owned stop is not.

Attribute 2 — taking ownership	Owning **the whole problem, not just the parts that go as planned**. It is not someone else's project that you are running; when something goes wrong you do not shrug and decide the work is impossible.	The named failure mode is **cover-your-ass engineering**: “it's not my fault, they broke it, they used it wrong, I built it to spec.” Mature engineers accept the responsibility given to them — and if they lack the authority to be held accountable, they seek ways to fix that.	Ownership means using your own judgement rather than constantly asking permission — but **not operating in private**. The better alternative to “seek forgiveness rather than permission” is **radiating intent**: signalling what you are about to do before you do it, which gives everyone context and creates an opportunity to intervene if you are about to do something dangerous. And it has an ownership property the permission-seeking route lacks — **the radiator keeps responsibility if things go sour; it does not transfer the blame the way asking permission does**. When a decision is needed, avoid staying on the fence: weigh the options, choose decisively, explain your reasoning, be willing to vote against your own preferences, and make the cost of being wrong as low as possible.
Asking “obvious” questions	Using your seniority to ask the questions that are **so obvious nobody else is willing to ask them** — because you have a responsibility to make the implicit explicit.	“You are planning to run a mission-critical service in a team of two — how will on-call work?” “I assume you have evaluated what it would take to move off this old system rather than keeping it alive?” “What happens if users start depending on the field you are telling them to ignore?”	vs pedantry. The asymmetry is the whole point and it is explicitly called unfair: **if a junior person asks these, the team may sigh and say yes, obviously we thought of that; if an expert asks, team members learn to include explicit answers in their design documentation** — or genuinely consider the question for the first time. The same asymmetry applies during incidents: it is **safer for senior people to admit they do not know**, so when raw facts are being reported that nobody can interpret, you are the one who should say so.
Glue work	The leadership and administrative tasks that are **not on anyone's job ladder but are needed to make a team successful**: unblocking, onboarding, reminders, mentoring, scheduling.	Projects cannot succeed without it, and it is rarely rewarded or allocated fairly — **it falls to whoever on the team cannot look away from the problem, often a junior person with a strong sense of ownership**. Leaders frequently do not step in, because the work is needed and they are glad it is getting done.	vs delegation. The harm is career-shaped: junior people doing too much of it **spend their prime technical learning years on work that does not teach them technical skills**, which can stunt them in the long run. Worse, the same work is read differently — **managers, promotion committees and future employers may count it as leadership from a senior engineer and dismiss it from a junior one**. So the instruction inverts the usual one: take the work that is nobody's job, and **redirect your junior colleagues toward tasks that will develop their careers**. Meeting notes are the canonical example, and they are also a lever — you record the facts you think matter, document decisions, are first to frame them, then invite everyone to confirm.

Taking charge	Seeing a gap and stepping up to fill it. **It does not require prior authority.**	In an emergency there are likely to be many responders, and unless they work together the chaos makes the problem worse. But **coordinating only works if everyone knows you are coordinating** — if you do not take charge explicitly, you are one more person making noise. Better still, have the roles agreed in advance; the **incident command system** is the standard choice. Then make the coordination valuable: take clear notes, make sure everyone has the same context, and ask everyone to radiate intent.	vs seniority asserting itself. It also covers the awkward case: when someone says something disrespectful in a public channel, the usual “praise in public, criticise in private” rule is explicitly reversed, because **if it looks like the original message was not addressed, that kind of message becomes acceptable** — and if someone was attacked in front of a group, they need support in front of the same group. You do not have to say the perfect thing; there often is not one. Useful moves: describe the culture you are aiming for and use it as the reference, and give the person a path to being on the same side as you.
Creating calm	Being the steady one: **if you are dealing with a big problem, try to make it smaller; if it is a small problem, keep it small**.	When someone brings you a fraught situation, ask questions and understand why they are telling you — do they need to vent, or do they want action? Even seeing that you have the same information and are not panicking can reduce a colleague's anxiety. **A senior person making a fuss can blow a minor thing into a big loud issue**, so if joining in would amplify rather than calm, consider staying out.	vs suppressing problems. It has three specific parts. **Where your worries go**: not onto more junior people — it is not fair to ask them to carry your concerns — and when you do vent upward or sideways, be clear whether you want action or are blowing off steam, or the other person may reflexively amplify it. **Avoiding blame**: a big outage is an expensive training course, so if you are paying the cost you had better all learn something — the questions that make that possible are what exactly happened, what factors led them down this path, was there information they did not have but could have had, and where did their mental models diverge from reality. **Being consistent**: the wild-card leader nobody can prepare to meet is the anti-pattern, and consistency requires working sustainably, which you are also modelling.

Attribute 3 — remembering the goal	Holding on to the fact that **the software is the means, not an end in itself**: there is a business, a budget, a user and a team.	**Business:** adapt to the situation — sometimes a faster solution is better and sometimes a more stable one; during an outage the usual principles may go out of the window, and the senior move is to get the system back online first and clean up later. **Budget:** do not obsess, but know what kinds of expenditure or saving count as big, how the company makes money, and whether these are good times or lean ones. **Resources:** headcount is finite and staffing a project has an opportunity cost; be judicious with your **innovation tokens** — the limited capacity to do something creative, weird or hard. **User:** know who uses your software and how, and note the verdict — **whether or not you built it to spec, if you did not make your user happy you built the wrong thing**. **Team:** do not become a single point of failure.	vs commercial-mindedness as a personality. Each of these is a constraint that changes technical decisions. Two of them contain a redefinition worth memorising. Build the most useful thing, **not the thing that would be most fun to build** — and when it is time to stop polishing and call it good enough, stop. And on the team: **if you can reach the goal by empowering someone else to do better work, that is just as much a victory as solving it yourself**, because your impact is **what would not have happened without you**, not what you personally did.
Attribute 4 — looking ahead	Planning past the current moment, on the basis that the code and architecture you work on are likely to still be in use in **five or ten years**, and the systems around them much longer.	Ask what Future You will wish Present You had done. **Telegraph what is coming** — you can announce the intention to deprecate something long before you are ready to run the migration, and new projects will know not to invest in it while some teams may move on their own. **Tidy up** — leave the codebase and documentation so they just work for whoever comes next, and **leave no traps**, such as dangerous scripts everyone must remember not to run. **Keep your tools sharp** — every minute off your time to detect a problem or deploy a fix is a minute off every future outage. **Create institutional memory**, since eventually you will have complete staff turnover: decision records explaining what you were thinking, diagrams that include the obvious things everyone knows, and searchable keywords.	vs over-engineering, which is a different failure covered in Part 3. The distinguishing test is that these actions cost little now and compound later. Also here: **expect failure** — you cannot predict everything that will go wrong but you can predict that something will, so test the error paths as thoroughly as the success paths, practise the response with drills or controlled outages, and if you have not tested restoring your backups, **assume you do not have any**.

Optimising for maintenance, and building to decommission	Software is created once and **maintained for years**, so the thing to optimise is the long tail rather than the moment of creation.	**The system will never again be as well understood as it is on the day it is created** — expect that knowledge to decay a little every day. You have two options for future understanding: run continual education so everyone who might work on it is trained, or make it as easy as possible to understand when needed — a clear short introduction, at least one big simple picture with arrows showing which way the data moves, links to anything they might wonder about, and internals exposed through tooling, tracing and useful status messages. And **keep it simple**: it is easy to make something complicated and much harder to make it simple.	The method for simplicity is concrete rather than exhortation: **treat your first solution as "just the first" and spend at least the same amount of time again**, now that you understand the problem better — fewer lines, fewer branches, fewer teams, fewer running components, fewer files touched. The longer the system is meant to last, the longer you should spend simplifying. Where complexity is genuinely irreducible, **decide deliberately where in the system to put it**, so the rest can be reasoned about and there is one place to go. Finally, **build to decommission**: imagine you personally must turn this off in ten years, and add the clean interface, the ability to see which clients still use it, and a little distance between systems being integrated. **The side effect of designing for decommissioning is something modular and easy to maintain.**

> **CONCEPT BOUNDARY — THIS IS A CLAIM ABOUT CAUSATION, NOT A CODE OF CONDUCT**
>
> **What it sounds like:** a list of virtues. Be good, be calm, be honest, care about the business, think long term — the sort of advice that is agreeable, unfalsifiable, and changes nothing on Monday.
> **What it actually is:** a set of predictions about other people's behaviour, each with a stated mechanism. Approve a change without reading it, and reviews across the team get shallower — not because anyone decided to, but because what senior people do is what correct looks like. Bluff in a meeting rather than admit you do not know something, and the more junior people in that meeting will bluff too, and the shared model everyone is trying to build gets slower and worse. Take the note-taking and the unblocking yourself, and a junior engineer spends that hour on technical work instead of on a task nobody will count in their favour. Every item here is of that shape: an action of yours, and a predicted change in what other people treat as normal.
> **Why that matters practically:** it tells you which items are load-bearing when you are short of time. Admitting ignorance, owning a mistake in public, and taking the unglamorous work are high-leverage precisely because they are the behaviours people least expect to see from someone senior, and therefore the ones that move the norm the furthest.
> **The trap:** reading the list as a bar to clear. These are stated as aspirational qualities and ideals rather than a test of whether you are a real engineer — and the material is explicit that best practice depends on the situation. **If the advice contradicts your own judgement, trust yourself**: knowing when the conventional wisdom is wrong is itself part of the job.

Anchor sketch — the four attributes, and what each one predicts

Legend: bold = a named attribute or practice **reversed = the fact everything else answers** **Q = cover the page and answer this**

YOUR WORK IS CHECKED LESS AND COPIED MORE.
people stop guiding you and look to you for guidance.
so HOW YOU BEHAVE IS HOW OTHERS WILL BEHAVE --

this is PASSIVE influence. it is not opt-in.
VALUES ARE WHAT YOU DO. the clearest signal of what a
company values is WHO GETS PROMOTED. principles about
careful review die the day a senior person rubber-stamps.
|
FOUR ATTRIBUTES -- aspirational, not a bar to clear.
if the advice contradicts your judgement, TRUST YOURSELF.
---

1. BE COMPETENT

KNOW THINGS . experience = time + exposure + study,
NOT aptitude. staff ~10 yrs, principal ~15.
do not rush past your prime learning years.
stay anchored in the tech, not only in how your
company operates.
"NOT TECHNICAL ENOUGH" describes a PERSON, not a skill,
and has no path out. be precise: years? domain?
which skills?
BE SELF-AWARE . admit what you know (do not set the bar
at best-in-industry) AND what you do not.
bluffing slows the shared model everyone is
building. seniors admitting ignorance makes it safe.
explaining plainly IS the indicator of expertise.
HIGH STANDARDS . in ALL of it, not the fun parts. duct
tape is a decision about the PROBLEM, not about
what is interesting.
OWN MISTAKES . admit, then fix. never let someone else
be blamed. openness here is a psychological-safety
boost for the whole team.
BE RELIABLE . "X will be there, so I need not go."
finish what you start -- or stop deliberately and
SAY SO.

---

2. BE RESPONSIBLE

OWNERSHIP . the WHOLE problem, not just the parts that
go to plan. the anti-pattern: cover-your-ass
engineering ("I built it to spec").
RADIATE INTENT -- signal before you act. gives
others a chance to intervene AND keeps the
responsibility with you (permission transfers it).
DECIDE. off the fence. explain the reasoning.
be able to vote against your own preference.
ASK OBVIOUS QUESTIONS . unfair but true: a junior asks
and the team sighs; an EXPERT asks and it goes in
the design doc next time.
GLUE WORK . needed, unrewarded, falls on whoever cannot
look away -- usually a junior with ownership.
it is read as LEADERSHIP from you and DISMISSED
from them. so TAKE IT, and send them to the
technical work.
TAKE CHARGE . no prior authority needed. in an emergency
coordination only works IF PEOPLE KNOW you are
coordinating. agree the roles in advance --
the incident command system.
and if something is said publicly that should not
be: answer it PUBLICLY. the usual "criticise in
private" is reversed here.
CREATE CALM . big problem -> smaller. small -> keep it
small. do not pour your worries on juniors. be
clear if you want action or are venting.

```
          AVOID BLAME: what happened . what led them there .
          what did they not know but could have . where did
          their mental model diverge.
          BE CONSISTENT -- which means working sustainably.
-------------------------------------------------------
3. REMEMBER THE GOAL . software is the MEANS.
     BUSINESS . adapt. in an outage, restore first, tidy after.
     BUDGET . know what counts as big, and lean vs good times.
     RESOURCES . headcount is finite; staffing has an
          opportunity cost. spend INNOVATION TOKENS wisely.
          build the most USEFUL thing, not the most fun.
     USER . spec or no spec, an unhappy user means you built
          the wrong thing.
     TEAM . do not be a single point of failure. your impact
          is WHAT WOULD NOT HAVE HAPPENED WITHOUT YOU.
-------------------------------------------------------
4. LOOK AHEAD . this code may run in 5-10 years.
     TELEGRAPH . announce the intent to deprecate long before
          you can migrate. new projects stop investing.
     TIDY UP . leave no traps.
     SHARP TOOLS . a minute off detection is a minute off
          EVERY outage.
     INSTITUTIONAL MEMORY . turnover is total eventually.
          decision records; diagrams with the "obvious"
          things; searchable keywords.
     EXPECT FAILURE . you cannot predict WHAT, only THAT.
          practise the response. untested backups are
          not backups.
     MAINTENANCE > CREATION . the system is never better
          understood than on day one.
          KEEP IT SIMPLE: treat your solution as "just the
          first" and spend the same time again. put the
          irreducible complexity in ONE deliberate place.
     BUILD TO DECOMMISSION . assume YOU must turn it off in
          10 years. the side effect is modular and
          maintainable.
-------------------------------------------------------
THE METRIC: whether other people WANT TO WORK WITH YOU.
```

Q Name the four attributes and the three things competence is built from.

Q Why is glue work a career hazard for a junior engineer and a credit for a senior one?

Q Give the two properties radiating intent has that asking permission does not.

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Saying “I don't know” in a room where you are the expert	Speed — the shared model gets built honestly — and permission for everyone junior to do the same.	A moment of exposure, in a culture that may still treat not knowing as a defect.	Whenever it is true. It is safer for you to ask than for anyone more junior, which is exactly why it should be you.
Radiating intent instead of asking permission	Context for everyone, a chance for them to stop you, and the responsibility staying with you.	Visibility of your plans before they are polished, and the occasional intervention you did not want.	Whenever you are about to act on your own judgement in a way others will feel.

Taking the glue work yourself	The project functions, and your junior colleagues spend their prime learning years on technical work.	Hours that produce nothing on your own technical record, and work that is invisible when it goes well.	When it is needed for the project and the person currently absorbing it is too junior to afford it.
Taking explicit charge in an emergency	Coordination that actually coordinates, rather than one more voice in the noise.	Accountability for the response, and the exposure of doing it in front of everyone.	When responders are working past each other. Announce it, or it does not count.
Writing code as a senior engineer	Understanding, credibility, engagement, and feeling the cost of your own decisions.	It is rarely the highest-leverage use of your time, and most of it could be done by someone more junior.	When it teaches you something you cannot get otherwise, or sets a pattern others will copy — not when it is the comfortable option.
Spending as long again simplifying a working solution	Fewer lines, fewer branches, fewer components, and a system people can still reason about in two years.	Double the design time on something that already works.	In proportion to how long the system is intended to last. Short-lived things do not earn it.
Designing so the system can be decommissioned	A clean cutover for whoever turns it off — and, as a side effect, something modular and easy to maintain now.	Interfaces and separation you do not strictly need today, and a slower integration.	Whenever you are wiring in something new that others will build on top of.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“I never told anyone to skip the tests.”	“I skipped them once under pressure, so now that's the standard. Values are what you do.”
“I didn't want to look like I didn't know.”	“It's safer for me to ask than for anyone junior. If I bluff, they'll bluff, and everything slows down.”
“They're just not technical enough.”	“Which is it — hands-on years, this domain, or a specific skill? The vague version leaves them no way forward.”
“I'll just do it and apologise later if it's wrong.”	“I'll radiate intent: say what I'm about to do first. They get a chance to stop me, and I keep the responsibility.”
“It broke, but I built it to the spec I was given.”	“That's cover-your-ass engineering. I own the result, so what's the fix and who needs to know now?”
“Our newest engineer has been great at organising everything.”	“They're carrying the glue work, and it won't count for them the way it counts for me. I'll take it and give them the technical work.”
“Everyone's already responding to the incident.”	“Nobody has said they're coordinating, so we're just noise. I'm taking the incident command role — here's what I'll do.”
“Someone made a mess and we need to find out who.”	“What information did they not have but could have had, and where did their mental model diverge from reality?”
“This is the most elegant design I've come up with.”	“That's the first one. Now I spend the same time again seeing whether it can be simpler.”
“We'll worry about turning it off when we get there.”	“Assume I personally have to decommission it in ten years. What interface do I want to have left myself?”

“I solved four hard problems this quarter.”	“What wouldn't have happened without me? Empowering someone else to do it well counts the same as doing it.”

PART 4 · QUESTION 1

A senior engineer joins a team of eight. Within a quarter: they merge their own changes without review because they are “obviously safe”, and the team's review turnaround quietly halves in depth; in an architecture meeting they are asked about a storage engine they have never used and describe it confidently from half-remembered documentation; a configuration change of theirs causes a two-hour outage and, in the review, they explain that the deploy tooling should have caught it and that a junior colleague had approved the change; and the team's newest engineer has become the person who runs the standups, chases the release checklist, onboards contractors and takes every meeting's notes, which the senior engineer describes approvingly as “great ownership”. Nobody has complained about any of it.

(a) For each of the four behaviours, name what it models and give the predicted effect on the people watching. **(b)** The storage-engine answer has two separate costs beyond being wrong. Name both, and give the reason it is specifically safer for this person to say they do not know. **(c)** Rewrite the outage response as it should have gone, naming the three obligations it must satisfy. **(d)** Explain why the newest engineer's situation is a hazard rather than a success, name the work, and say what the senior engineer should do instead.

PART 4 · QUESTION 2

A platform team is about to ship a scheduling service. It is the most sophisticated thing they have built: a bespoke rules engine, a custom coordination protocol, and machine-learned placement — chosen, the lead says, because “a simple version would have been embarrassing”. It hard-codes calls into three other services' internals, since that was fastest. There is no runbook and no diagram; the two authors say they will explain it to anyone who asks. Backups are configured and have never been restored. The company has just entered a hiring freeze and asked every team to justify its spend, and the lead's justification is that the service is technically impressive. During the launch incident that follows, the lead sends increasingly alarmed messages to the whole engineering channel and asks three junior engineers whether they think the company will cancel the project.

(a) Name the two long-horizon principles the design violates, and give the concrete method offered for the first of them. **(b)** The hard-coded calls will cost someone specifically. Name the practice that answers this, state the test it uses, and give its side effect. **(c)** Take the documentation, the runbook and the untested backups together: name the two options for making a system understandable later, say which one this team has implicitly chosen, and give the stated position on the backups. **(d)** Evaluate the justification and the lead's behaviour during the incident against the relevant attributes, naming what should have been done in each case.

2×2 — what you are good at, and what people learn from you

COLUMNS: DOES YOUR BEHAVIOUR SET A STANDARD PEOPLE CAN SAFELY COPY? →

	Yes — admits ignorance, owns mistakes, stays calm, takes the unrewarded work	**No — bluffs, deflects, amplifies, takes the interesting work**
TECHNICALLY strong and reliable	**A role model in the useful sense** The work is good and so is the example, so the standard spreads without anyone having to enforce it: people ask questions, mistakes surface early, and the unglamorous work gets done by whoever can best afford it. *Move:* the cell is not a resting place, because the behaviours that got you here are the ones under most pressure when you are busy. The three that keep it are the cheapest: say you do not know, own the mistake in public, and leave the systems and documents so the next person just works.	**Load-bearing and quietly corrosive** Excellent work, and everything around it degrades: reviews go shallow because yours do, nobody admits confusion because you never have, and small problems arrive already large. *Move:* nothing here requires being better at engineering. Ask one obvious question in the next design review, say “I don't know what to do with that information” when it is true, and when the next thing breaks, be the first to say which part was yours.
STILL BUILDING the technical depth	**Trusted, and not yet a technical leader** People like working with you and learn good habits from you, and you cannot yet underwrite the technical decisions — and no amount of leadership substitutes for the “technical” in technical leadership. *Move:* invest deliberately in time, exposure and study rather than waiting for aptitude, and be wary of a role that leaves you learning only how your company operates. Your general experience is a hook: pattern-match the new domain's shapes to what you already know.	**Neither yet** The honest starting position for most people, and a fine one — provided it is named. The risk is only that the vague label “not technical enough” gets attached to it, which describes a person rather than a skill and offers no way out. *Move:* convert the label into its precise versions — hands-on years, this domain, or specific named skills — because each of those has a remedy. Then start with the behavioural half, which is available immediately and costs nothing.

Read it as: the row is what you can do and the column is what the people around you conclude is normal, and the reason both are here is that they are not the same axis and only one of them is on your performance review. The top-right cell is the interesting one: it is the strongest engineer in the room quietly lowering the standard of everyone else's work, at no cost to their own output, which is why nobody reports it and why it can run for years.

CARRY-OUT RULE FROM THIS PART

Assume you are being copied, and ask what the last thing you did would look like as a rule. The standard is set by what you actually do under pressure, not by what you would defend in principle. Three behaviours move it furthest and cost least: admitting what you do not know, owning your own mistakes out loud, and doing the work that helps the team and counts for nobody. And the test to keep at the end of all of it — **the metric for success is whether other people want to work with you**; if they do not, reconsider the approach.

SECTION 6

Concept sketch — the whole subject, end to end

Four named groups, in the order the story runs: **THE VIEW → THE GROUND → THE WORK → THE EXAMPLE**. Every node carries a question. Cover the answers, walk the map, and each answer hands you the next question.

Legend: bold = a group heading **reversed = the one group to remember** **Q = retrieval cue**

The four groups run in order: **THE VIEW** — what you can see · **THE GROUND** — what you must cross and where you are going · **THE WORK** — turning that into a project · **THE EXAMPLE** — what you produce while producing it.

GROUP 1 - THE VIEW . depth is paid for in perspective

THE MAPS ARE OBSCURED, NOT MISSING. clearing them is a task with a plan. and they FOG BACK UP.
Q what makes context-gathering a task rather than a trait?

KNOWING THINGS = SKILL (paying attention) + OPPORTUNITY (access). neither substitutes for the other.
Q two equally observant people, different pictures -- why?

FOUR RISKS OF DEPTH
PRIORITISING BADLY -- the local maximum inflates
LOSING EMPATHY -- fish-eye lens; other domains look easy
TUNING OUT -- you stop seeing what is broken nearby
FORGETTING WHAT IT IS FOR -- the goal died, work goes on
Q which two are mirrors, and what single cause makes both?
Q which one ends with "nobody owns the end result"?

FIVE WAYS TO SEE BIGGER outsider view . escape the echo chamber . what is actually important (+ THE OBJECTIVES THAT ARE ALWAYS TRUE) . the customer's view . prior art
Q which objectives are stated only when they are in danger?
Q what is the test for whether your work wastes your time?

GROUP 2 - THE GROUND . terrain, and the place you are going

BEING RIGHT IS LESS THAN HALF THE BATTLE. you must also make people CARE -- and ask in the right month.
Q what dull mechanism makes everything take a quarter longer?

TEAM TECTONICS . reorgs cut communication; loaded teams entrench; a new leader is an earthquake.

SEVEN CULTURAL ATTRIBUTES secret/open . oral/written . top-down/bottom-up . fast/deliberate . back channels/ front doors . allocated/available . liquid/crystallized
Q published values vs real values -- which is which?

POWER, RULES OR MISSION . pathological / bureaucratic / generative, across seven rows ending in how FAILURE and NOVELTY are treated.
Q what happens to messengers in each of the three?

FIVE POINTS OF INTEREST, FIVE DIFFERENT REMEDIES
chasms -> bridge . fortresses -> sponsorship or the password (most gatekeep because they CARE) . disputed territory -> one owner for the whole . deserts -> evidence this time differs . paved roads and goat tracks -> which one do people use?
Q what does going the long way round a fortress cost you?

THE ROOM . to set direction you must be an insider. but admitting you is NOT free: argue ORGANISATIONAL impact, and reduce the cost of your presence.
Q three reasons the room may not be what you think?

THE SHADOW ORG CHART . connectors and old-timers. the director who goes cold has consulted someone.

THE TREASURE MAP . define the treasure = define success. FIVE COSTS OF SHORT-TERM ONLY: one direction is hard . big things never finish . CRUFT . competing initiatives

. engineers stop growing
Q the two branches by which cruft accumulates?
THE TECH TREE . you research physics because you want a railroad. the real goal is TIME TO MARKET.
Q what can you evaluate once you know the real goal?

GROUP 3 - THE WORK . ambiguity, not technology

NOBODY REPORTS TO YOU AND YOU OWN THE RESULT -- including the fissures between teams and the work that is nobody's job.
OVERWHELM IS NORMAL. "that feeling of discomfort is called learning." FIVE: anchor doc . the sponsor conversation . who gets your uncertainty (NOT the juniors) . give yourself a win . use your strengths
Q why is the START the cheapest time not to know things?
EIGHT POINTS OF CONTEXT goals . customer needs . success metrics . sponsors and stakeholders . fixed constraints . risks . history . team
Q which one can make you reject the premise of the project?
Q why does "how much code is written" fail as a metric?
STRUCTURE roles (RACI; ONE accountable) . scope (the triangle; MILESTONES AS BETA TESTS) . estimation (only experience ON that project) . logistics . kickoff
Q the two failure patterns role definition exists to break?
DRIVE choose the route, decide, react to hazards.
EXPLORE -> if the goals are just a description of your implementation, you have not explored yet.
CLARIFY -> models, naming, pictures. you have the incentive to understand; your helpers do not.
DESIGN -> implicit agreement means nothing. a written design is a VERY CHEAP ITERATION. five minimum headings. WRONG IS BETTER THAN VAGUE.
PITFALLS -> ten, incl. "this looks easy" (if it seems trivial you do not understand it), "we might need it later" is not a justification, 3 p.m. vs 3 a.m., BIKESHEDDING.
COMMUNICATE -> impact, not activity. no WATERMELONS.
NAVIGATE -> something WILL go wrong; reroute, escalate, or say the goal is unreachable.
Q what makes a project lead a bottleneck?

GROUP 4 - THE EXAMPLE . checked less, copied more

PASSIVE INFLUENCE IS NOT OPT-IN. how you behave is how others will behave. VALUES ARE WHAT YOU DO.
Q what is the clearest indicator of what a company values?
BE COMPETENT know things (time+exposure+study) . self-aware in BOTH directions . high standards . own mistakes . be reliable
Q why is it safer for a senior person to admit ignorance?
BE RESPONSIBLE ownership (vs cover-your-ass engineering) . RADIATE INTENT . decide . ask obvious questions . take the GLUE WORK . take charge explicitly . create calm
Q what does radiating intent keep that permission gives away?
Q why is glue work a hazard for a junior and a credit for you?
REMEMBER THE GOAL business . budget . innovation tokens . user (spec or not, unhappy = wrong thing) . team
Q how is your impact defined, if not by what you did?
LOOK AHEAD telegraph . tidy . sharp tools . institutional memory . expect failure . maintenance over creation . keep it simple . BUILD TO DECOMMISSION
Q the concrete method for making a design simpler?
Q the side effect of designing for decommissioning?
THE METRIC: DO OTHER PEOPLE WANT TO WORK WITH YOU?

Master 2×2 — the decision card you can carry to other problems

The two questions that survive when the vocabulary is stripped away: **can you see the whole picture** — where you are, what the ground is like, where it is all going — and **are you acting on it deliberately**, both in the work and in the example you set while doing it?

COLUMNS: ARE YOU ACTING ON IT DELIBERATELY? →

	Yes — route chosen, decisions made, behaviour treated as output	**No — reacting, and assuming the example takes care of itself**
You can see POSITION, TERRAIN and DESTINATION	**Leading at scale** You know how big your part of the world is, how this organisation actually moves, and what the destination is — and you convert all three into chosen routes, named deciders, written designs, and a standard people can copy. *Move:* the maintenance is the job. All three views decay: priorities re-rank, a reorganisation redraws the terrain, and the destination gets reached or abandoned without announcement. Keep one relationship outside your group, one standing source of organisational news, and one sentence stating what the current work is a step toward.	**The excellent observer** You can describe the politics, the gaps and the long-term goal better than anyone, and none of it turns into a route. Your accuracy becomes commentary, and your behaviour under pressure teaches whatever it happens to teach. *Move:* pick the smallest deliberate act available and do it — name the decider, write the design down, send the summary nobody is sending. The map's stated purpose is to go where the dragons are, not to admire them from a distance.
You see your OWN PATCH only	**Efficient in an unknown direction** Strong execution, clear decisions, good habits modelled — aimed by whatever is nearest. The local maximum feels like the summit and the project outlives the goal it was for. *Move:* the correction is other people rather than more thinking. Ask the newest person what they see, talk to peers outside the group, find out how the business ranks things right now, and ask what this work was originally for and whether that is still true.	**Busy** No view and no steering: overwhelm never resolved, obstacles met for the first time by everyone who meets them, and a standard set by accident. *Move:* do not attempt everything. Two cheap things first — an anchor document for yourself, and one conversation with whoever is sponsoring the work, carrying a written guess at what they want and what success would look like. Everything else follows from having those two written down.

Read it as: the row is *sight* and the column is *agency*, and the whole subject is the claim that seniority fails at one or the other rather than at engineering. The same pair separates a surveyor who can describe a route from an engineer who builds the road, a doctor who can name the condition from one who changes the treatment, and a critic who can explain what a piece needs from the person who can rewrite it.

THE THUMB RULE

Past one team, the binding constraint is what you can see and whether you act on it — and while you act, you are the standard. All three halves get skipped for different reasons: perspective helps nobody today, terrain looks like politics, and the example feels like something other people worry about.

SECTION 7

Symptom → diagnosis → move

Cover the middle and right columns and work down the left.

Getting oriented

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
Every person you talk to agrees with you.	An echo chamber. Your beliefs are untested rather than confirmed.	Build relationships with senior peers outside your group, and go beyond engineering. Ask what other teams say about yours.
Your team's problems feel special and worthy of unique solutions.	Prioritising badly — the local maximum inflating with time spent staring at it.	Check prior art inside and outside. The genuinely brand-new problem is the unusual case, not the default.
You catch yourself saying another team's problem could be done in a weekend.	Loss of empathy. A fish-eye lens makes your domain rich and everything else flat.	Go and learn a little of their domain. Notice that this also shapes your words and the motives you assign to people.
A new joiner asks why something obviously broken is like that, and you have no answer beyond “it always has been”.	Tuning out the background noise. It stopped registering months ago.	Treat their question as data, not naivety. Ask what else they can see while they still can — and hold that view yourself once they cannot.
Nobody can say what the project is ultimately for.	Forgetting what the work is for. The goal may be dead or solved elsewhere.	Trace it up: what goal did this serve, is it live, and has anything else met it? This is the risk with the highest cost per month.
You are consistently surprised by decisions that affect you.	You do not know where decisions are made, or who influences them.	Ask a trusted colleague to walk you through where a specific decision came from — clear that you are not fighting it, only learning the mechanism.
Your availability figure is good and your users are unhappy.	You are measuring from your side of the boundary.	Measure from the user's point of view. They do not distinguish the parts of the chain; either it worked for them or it did not.

Moving an idea, or a project

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
Your proposal is technically unanswerable and nothing happens.	Being right is less than half the battle — nobody has been made to care, or you asked in the wrong month.	Find who must care and why it serves them. Check the planning cycle; just after objectives are set is the worst moment.
A senior person is enthusiastic, then goes quiet.	The shadow org chart. They have consulted an influencer you have not convinced.	Find the connectors and old-timers and convince them. No amount of extra document will substitute.
One team or person blocks every route.	A fortress — and most gatekeep because they care about quality and safety.	Bring a token of sponsorship they respect, or work out the password. Going round costs you their wisdom as well as time.
Nobody can tell you whether a change to a system is safe.	Disputed territory: ownership smeared across teams, none able to speak for the whole.	Establish one owner for the whole system before proceeding — or accept that you must build the context yourself, and budget for it.
Everyone discourages an initiative because it has failed before.	An uncrossable desert.	Not a prohibition, but a requirement: bring evidence that this attempt is different, or pick a different route.

Newcomers keep taking weeks to do something that regulars do in a day.	The official paved road is not the road people use; there is a goat track.	Learn the goat tracks and decide whether to document them, weighing how soon the paved road will really be fixed.
Every decision keeps being reopened.	No agreed decider, or no process for deciding.	Define roles explicitly. The value of a formal matrix is that it is uncontroversial enough to raise without it being weird.
The goals document names a technology in its first line.	Exploration was skipped; the implementation has become the goal.	Go back and frame the problem. Target: state each team's want in a way they will agree is accurate, plus what you are not doing.
Everyone nods at the design and later builds something else.	Implicit agreement, which may not mean anything.	Write it down. A written design is a very cheap iteration, and the review covers whether it is the right problem at all.
A review spends most of its time on the least consequential item.	Bikeshedding: the trivial issue is easier to discuss than the hard one.	Name it and redirect. Reserve the hard question explicitly, and be honest about whether the trivial one is reversible.
Status is green every week until it suddenly is not.	A watermelon project.	Report in impact and raise the risk to a key milestone as soon as it is real. A delay that does not move the date may not need raising at all.

Leading, and being watched

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
Review quality across the team has quietly declined.	Values are what you do. Someone senior is approving without reading, and it has become the standard.	Fix the behaviour, not the policy. What you do is what others will treat as correct.
Nobody in meetings ever admits confusion.	Senior people are bluffing, so nobody else can afford not to.	Say you do not know, out loud, when it is true. It is safer for you than for anyone junior, which is why it should be you.
Many responders, no progress, during an incident.	Coordination that nobody has announced is not coordination.	Take charge explicitly, then make it valuable: clear notes, shared context, everyone radiating intent. Better still, agree the roles beforehand.
Raw metrics are being reported that nobody can interpret.	Everyone is confused and nobody senior enough has said so.	Say “I don't know what to do with that information”. Ask what is typical, what the theory is, and how it fits.
Your most junior engineer runs the standups, the checklist and the notes.	Glue work landing on the person who can least afford it.	Take it yourself — it reads as leadership from you and is dismissed from them — and give them the technical work instead.
A mistake is followed by an explanation of why it was not really anyone's fault.	Cover-your-ass engineering, and the loss of the lesson everyone just paid for.	Own your part plainly, protect anyone at risk of being blamed, and ask the four blameless questions.
A minor issue has become a company-wide drama.	Amplification. A senior person joining loudly makes a small problem large.	Make big problems smaller and keep small ones small. If joining in would amplify, stay out — and keep your anxieties off junior people.
Nobody knows what to expect from you week to week.	Inconsistency, which removes the safety your presence is supposed to provide.	Be predictable about what you care about — and work sustainably, since consistency fails first when you are beyond capacity.
Only two people understand the new system, and they are happy to explain it.	The education option, chosen by default. Understanding decays from day one.	Switch to the other option: short clear introduction, one big picture with arrows, links, and observable internals.

Replacing a component means unpicking it from three others.	It was not built to be decommissioned.	Add the clean interface and the ability to see who still uses it. Assume you personally will have to turn it off in ten years.

Reverse mapping — you see a practice, what is it there for?

The other direction. Use this when auditing how a person or an organisation works rather than planning one move: for each thing you find them doing, say what it was buying and what it does *not* buy.

WHAT YOU FIND PEOPLE DOING	WHAT IT COUNTERS	WHAT IT DOES *NOT* COVER
Asking the newest joiner what looks wrong	Tuning out the background noise, and the loss of the outsider view as you become an insider.	Calibration. They can see the defects; they cannot tell you which of them the business currently cares about.
Standing relationships with peers in other groups	The echo chamber, and the negative view other teams hold of yours that nobody would otherwise tell you.	Access. Knowing more people does not put you in the room where decisions get made.
Skip-level conversations and all-hands attendance	Ranking your work against what the business needs right now, including the objectives that are only stated when in danger.	The terrain. Knowing the priorities does not tell you who blocks what or how a decision travels.
Mapping the culture on the seven attributes	Framing an initiative wrongly — an incremental pitch in a deliberate culture, or a full plan in a fast one.	Individual obstacles. A slider tells you the average journey, not that there is a fortress on your route.
Sending the summary nobody is sending	Chasms — the information gaps between organisations, and work falling into the unlined-up edges between teams.	Ownership disputes. A bridge carries information; it does not make anyone able to speak for a whole system.
A stated long-term destination	All five short-term costs at once: drift, unfinished big things, cruft, competing initiatives, and engineers who never grow.	Execution. Knowing the destination does not choose the route, staff it, or make anyone drive.
An anchor document at the start of a project	The overwhelm, and the “where did I write that down?” problem while your mental map is still forming.	Agreement. It is yours alone; nothing in it has been confirmed by the sponsor or anyone else.
A responsibility matrix such as RACI	Endless discussion with no decider, and decisions relitigated because there was no process for making them.	Whether the decisions are good. It settles who decides, not what is correct.
Milestones that are demonstrable	Building something nobody wants, and the absence of urgency when the only deadline is at the end.	Stability of the plan. Each usable increment can change the requirements for the next one — that is the price.
A written design with alternatives considered	Implicit agreement that means nothing, and solving the wrong problem for a whole quarter.	Delivery. A cheap iteration on paper is not progress on the thing.
Radiating intent before acting	Both failure modes of autonomy — acting invisibly, and asking permission in a way that transfers your responsibility to the approver.	The quality of your judgement. It surfaces the decision; it does not make it right.
A senior person taking the meeting notes	Glue work landing on someone junior, and the loss of a lever — the note-taker frames the decision first.	Their career development. Taking the work back does nothing unless you also hand them the technical work.

Announcing an intention to deprecate long before migrating	New investment in a system you know is going away, and a harder migration later.	The migration itself. Setting expectations is not staffing, and telegraphing is not deprecating.
Announcing what you are about to do before doing it	Acting invisibly, and the transfer of your responsibility to whoever grants permission.	Being right. Signalling the move gives people a chance to object; it does not improve the move.
A written definition of what you are *not* doing	Neighbouring teams assuming your project will also deliver the thing they actually care about.	Their disappointment. It relocates the conversation earlier; it does not make the answer welcome.
Naming one person accountable for a decision	Discussion that never converges and decisions that get reopened, because no process existed for making them.	Correctness, and buy-in. It says who calls it; it does not make the call right or popular.
Keeping a log of your own time estimates	Estimation that never improves, because the only reliable way to time a project is experience on that project.	Dependencies. Your own accuracy says nothing about whether the team you need has capacity this quarter.
A single documentation home for a project	The scattered-context problem: meeting recaps, milestone descriptions and objectives living in different places.	Agreement. One location makes the current answer findable; it does not make it correct or shared.
Practising the incident response before you need it	Disaster plans that have never been executed, and coordination invented under load by people who have not agreed roles.	Prevention. Drills tell you how you will respond, not whether the failure will happen.
Spending as long again simplifying a working design	Complexity that is easy to create, rewarded in some organisations, and paid for over the whole life of the system.	Understandability on its own. Fewer lines taken too far becomes obfuscation; the target is being able to reason about it.

SECTION 8

Numbers worth remembering

NUMBER	WHAT IT IS
3 maps	The locator map (where you are, and what is outside your scope), the topographical map (the terrain you must cross), and the treasure map (where you are all going). They already exist in your organisation; they are obscured rather than absent.
2 requirements	Knowing things needs a skill — paying attention, sifting facts out of the noise — and an opportunity — access to the decisions and discussions that affect your work. Neither substitutes for the other.
4 risks of depth	Prioritising badly, losing empathy, tuning out the background noise, and forgetting what the work is for. The first and third are mirror images produced by the same cause.
7 cultural attributes	Secret or open · oral or written · top-down or bottom-up · fast or deliberate change · back channels or front doors · allocated or available · liquid or crystallized. Most organisations sit somewhere in the middle of each; it is hard to manoeuvre at either extreme.
3 × 7	The typology of how organisations process information — pathological, bureaucratic, generative — separated across seven properties: orientation, cooperation, treatment of messengers, responsibilities, bridging, response to failure, and response to novelty.
5 points of interest	Chasms, fortresses, disputed territory, uncrossable deserts, and paved roads versus goat tracks. Each takes a different remedy, which is the reason for learning the list.
6 ways to stay current	Automated announcement channels · walking the floor · lurking (calendars, agendas, and the channel list sorted by most recently created) · making time for reading · checking in with your leadership · talking with people.
5 costs of short-term only	Harder to keep one direction · big things never finish · cruft accumulates · competing initiatives · engineers stop growing.
5 things that reduce overwhelm	An anchor document · the sponsor conversation · deciding who gets your uncertainty · giving yourself a win · using your strengths.
8 points of context	Goals · customer needs · success metrics · sponsors, stakeholders and customers · fixed constraints · risks · history · team.
4 letters, 1 accountable	Responsible, Accountable, Consulted, Informed — with only one accountable person per task, often the same person who is responsible.
3 in tension	The project management triangle: time, budget and scope. The blunt version is “fast, cheap, good: pick two”. If you have fewer people, you cannot do as much.
×3	The second most common piece of estimation advice — assume you are wrong and multiply everything by three. Neither it nor “break it into the smallest tasks” is very satisfying; the preferred answer is that the only way to time a project is by gaining experience on that same project.
5 minimum headings	What every design document should carry however small: context, goals, design, security/privacy/compliance, and alternatives considered or prior art. Often-useful additions: background, trade-offs, risks, dependencies, operations.
10 pitfalls	Brand-new problem · this looks easy · building for the present · building for the distant future · every user just needs to · the difficult part later · solving the small problem by making the big one harder · not really a rewrite · is it operable · discussing the smallest decisions the most.
3 p.m. vs 3 a.m.	The operability test: if you struggle to remember how something works at three in the afternoon, you will not understand it at three in the morning — and whoever joins after you have moved on will find it much harder.
>3, ideally 6+	The on-call staffing rule for a system going into production: more than three people, and ideally at least six, or you are setting the team up for burnout and dropped pages. Decide who it is and put it in the design document.

1957	The law of triviality, which holds that because a trivial issue is much easier to discuss than a difficult one, that is where teams tend to spend their time. Its example was a committee evaluating plans for a nuclear power plant that spent most of its time on the materials for the staff bicycle shed — hence “bikeshedding”.
2 influencers	The two kinds the shadow org chart identifies: connectors, who know people right across the organisation, and old-timers, who wield influence regardless of rank or title simply from having been around a long time.
8 / 10 / 15 / 20 years	The experience requirements published for successive grades by one long-established professional engineering body. Software is less rigorous, and most places do not consider years at all when allocating titles.
10 and 15	Typical years of experience for staff engineers and principal engineers respectively — a rough observation, not a rule.
7, and no fewer than 5	The advice on not rushing past your prime learning years: never accept a managerial role until you are solidly senior as an engineer, meaning at least seven years writing and shipping code and definitely no fewer than five. The same reasoning is applied to senior individual-contributor roles.
4 attributes	Being competent, being a responsible adult, remembering the goal, and looking ahead. They are stated as aspirational ideals rather than a bar to clear.
3 aspects of responsibility	Taking ownership, taking charge, and creating calm.
4 blameless questions	Exactly what happened · what factors led them down this path · was there information they did not have but could have had · where did their mental models diverge from reality.
5 or 10 years	How long the code and architecture you work on are likely to still be in use. The interconnected systems that make up a production environment may last much longer, and each component influences the ones that follow.
2 options	The only two ways future people will understand your system: continual education and hands-on training for everyone who might work on it, or making it as easy as possible to understand at the moment they need it. Most teams choose the first by default, without deciding to.
1 more time	The method for simplicity: treat your first solution as “just the first” and spend at least the same amount of time again seeing whether it can be simpler — fewer lines, branches, teams, running components and files touched. The longer the system is meant to last, the longer you should spend.
10 years	The decommissioning thought experiment: imagine that you personally will have to turn this component off in a decade, and that Future You will be no less busy than Present You.

Glossary

TERM	MEANING
Fog of war	The obscured — not absent — parts of your organisation's picture. Clearing it is a deliberate task, and it fogs back up as priorities change.
Locator map	Your place in the wider organisation: what is along the borders of your scope, and how large your part of the world is on a company-scale view.
Local maximum	The peak visible from inside one group, which starts to feel like the summit because everyone around you cares about the same set of things.
The objectives that are always true	Company needs so obvious they are only stated when in danger: continuing to exist, having enough money to pay everyone, having a good reputation.
Outsider view	Looking at your own group as if you were not in it. A new person can always see the problems, having missed the gradual change — and being new is not a licence to be a jerk.
Team tectonics	Domains of responsibility grinding against each other to form an organisational terrain of ridges, chasms and disputed ground, reshaped by reorganisations and new leaders.
Chasm	An information and culture gap between organisations, or between the unaligned edges of two teams' responsibilities, into which work and information fall.

Fortress	A team or person who blocks progress. Most are well intentioned and gatekeep because they care; the ways through are sponsorship, the password, a costly battle, or a long detour that loses you their wisdom.
Disputed territory	Work that several teams believe they own, or a system owned in pieces by several with none able to speak for the whole.
Uncrossable desert	A fight others consider unwinnable. Not a prohibition, but a requirement for evidence that this attempt differs.
Paved road / goat track	The official way that has been made the easiest way, versus the undocumented route people actually use because the official one does not go where they need.
The room where it happens	The forum in which decisions at your scope are actually made. Admission is not free to the people already there, so the case must be about organisational impact.
Shadow org chart	The unwritten structures through which influence flows — connectors who know everyone, and old-timers whose weight comes from tenure rather than title.
Bridging	Making connections between parts of an organisation that would otherwise have large information gaps: the summary nobody sends, the introduction nobody makes, the document showing how projects connect.
Treasure map	The shared destination and the milestones toward it, including a clear definition of success — and the story of where you came from as well as where you are going.
Technology tree	The image for why a piece of work is being done: capabilities form a graph, and much of what you build is an unavoidable step toward a goal you must not forget.
Anchor document	A document kept for yourself alone at the start of a project — an external part of your brain holding uncertainty, rumours, leads, reminders and lists.
Milestones as beta tests	Making every milestone usable or demonstrable, so users and stakeholders can react — accepting that each increment may change the next one's requirements.
RACI	Responsible, Accountable, Consulted, Informed. One accountable person per task. Its value is being an uncontroversial structure for raising who decides.
Wrong is better than vague	The preference for a specific claim that can be corrected over a hand-wavy one that cannot. Supported by naming the actor for every verb and putting a noun after “this”.
Bikeshedding	Spending disproportionate discussion on the most trivial and reversible decisions, because they are the easiest to grasp.
Watermelon project	A status reported green on the outside while red inside, which defers an unpleasant surprise to the end of the project.
Passive influence	The effect you have simply by how you act. At this level your work is checked less and copied more, so how you behave becomes what others treat as correct.
Cover-your-ass engineering	Deflecting responsibility onto the specification or the people who used the thing wrongly, rather than owning the result.
Radiating intent	Signalling what you are about to do before doing it — giving others context and a chance to intervene, while keeping the responsibility with you rather than transferring it to an approver.
Glue work	Unblocking, onboarding, reminders, mentoring and scheduling: needed for success, on nobody's job ladder, and read as leadership from a senior engineer while being dismissed from a junior one.
Innovation tokens	An organisation's limited capacity to do something creative, weird or hard — a budget to be spent deliberately rather than everywhere.
Build to decommission	Designing on the assumption that you personally will have to turn the system off in ten years. Its side effect is something modular and easy to maintain now.

Answer key

PART 1 · Q1 — THE PAYMENTS ENGINEER

(a) **The retry library. Prioritising badly.** When everyone around you cares about the same set of things it is easy to magnify their importance, and problems outside the group start to appear simple or unimportant by comparison; the more time you spend staring at your own group's problems, the more they seem special and unique and worthy of special, unique solutions. Sometimes they are — but **it is unusual to find a problem that is genuinely brand new**, and checking prior art means less time reinventing wheels. **The search comment. Losing empathy**: overfocusing until other technology areas look trivial next to your own rich, nuanced domain — a fish-eye lens that makes the thing in front of you huge and squeezes everything else into the periphery. It also silently shapes the words you use, what you explain versus leave implicit, and the motives you ascribe to other people. **The runbook. Tuning out the background noise**: work around the same broken process for months and you stop thinking of it as something that needs fixing; you may also fail to notice that something merely annoying has slowly become worse, so you cannot be objective about how quickly to react. **The answer to the director. Forgetting what the work is for**: in a silo you lose the connection to what is going on elsewhere, and a project can continue even though the goal no longer matters or has already been solved some other way; working only on your own part, you slip into a world where everyone does their little piece and **nobody feels responsible for the end result**. (b) **Prioritising badly and tuning out are the mirror images.** One says your group's concerns feel more important than everyone else's; the other says you stop noticing problems at all. The single cause of both is **time spent absorbed in one domain** — the same depth that makes the work rich and complex for you is what inflates your own concerns and makes the defects you live beside disappear. (c) The advice is the same error with the sign flipped. Caring less would not correct anything, because the failure is not one of attitude but of **accuracy**: depth reliably distorts the estimate of how much things matter, in both directions at once, and being generous about other teams does not recalibrate the instrument. The corrections are mechanical and external — look at where your group sits on the whole org chart, ask people outside it, find out what the business needs now. The test that replaces “care less” is specific: **it is fine not to be working on the most important thing, but what you are doing should not be a waste of your time, and if you cannot explain to yourself why what you are doing needs someone at your level, you may be doing the wrong thing**. (d) Any three of: **take the outsider view** — the new joiner just demonstrated that a new person can always see the problems, having missed the gradual change, so treat the runbook question as data and keep asking what they see; this attacks *tuning out*. **Check prior art** — study what has been done inside and outside before creating something new, remembering the goal is to solve the problem rather than necessarily to write code to solve it; this attacks *prioritising badly*. **Escape the echo chamber** — build relationships with senior peers in other groups and beyond engineering, including hearing what other teams say about yours; this attacks *losing empathy*. **Establish what is actually important** — all-hands for other groups, skip-level conversations, and the objectives that are always true; this attacks the ranking error. **Take the customer's point of view**, and **trace the project back to the goal it was serving** to check that the goal is still live; this attacks *forgetting what the work is for*.

PART 1 · Q2 — THE TOOLS TEAM

(a) Both are accurate because they are measured from different sides of a boundary. Availability figures are useful and they do not tell the whole story, because they do not settle **who defines “available”**. The users' complaints include a mirror the team does not own and builds that succeed slowly, and **a user does not distinguish between the parts of the chain** — at the end of the day there are things that work and things that do not. What they should measure is success **from the users' point of view**, which explicitly still applies when the customers are other teams inside the same company. If you do not understand your customer you do not have real perspective on what is important. (b) The novel-cache proposal is **prioritising badly** — the local maximum inflating until the team's problem feels special, unique and worthy of a special, unique solution. Two checks should precede it: **look for prior art**, since many problems are not essentially new and you will come up with better solutions by studying what other people have already done, inside and outside the organisation; and remember that **the goal is to solve the problem, not necessarily to write code to solve it** — take the time to understand what already exists before building something new. (c) The team has failed to establish **what is actually important right now**. Technology is a means to an end; you are there to help the organisation achieve its goals, and **the ordering changes over time**, so a plan must be made against this quarter's ranking rather than a remembered one — if customers are leaving because the product is missing things, it is not the moment to push internal work. Noticing when the goals change matters because it may mean your scope or focus should change. The category that a freeze and customer churn belong to is **the objectives that are always true**: continuing to exist, having enough money to pay everyone, having a good reputation — needs so obvious that **they are only really stated if they are in danger**, which is exactly what has just happened. (d) What is producing all three symptoms is **depth**. The same years inside the domain that made the lead expert have inflated their own problems, flattened the significance of everything outside, and detached the plan from the business — this is a predictable cost of focus rather than a character defect, which is why more diligence will not fix it and outside inputs will. The test to apply: it is fine not to be working on the most important thing, **but what you are doing should not be a waste of your time — and if you cannot explain to yourself why what you are doing needs someone at your level, you might be doing the wrong thing**.

PART 2 · Q1 — THE DEPLOYMENT-TOOL CONSOLIDATION

(a) **The security group is a fortress** — a team whose approval you need and whose time you cannot get. Most fortresses are well intentioned and gatekeep because they care about quality and safety. The remedies are a **token of sponsorship from someone the gatekeeper respects**, or the password: proving you have mitigated the risks of the change, completing the checklists or capacity estimates, answering the document comments acceptably. Going the long way round complicates the journey and **loses you whatever wisdom the gatekeeper would have shared**. **The two abandoned attempts make this an uncrossable desert** — a battle other people consider unwinnable, where any suggestion of trying again meets discouragement and ennui. That is not a prohibition; the requirement is **enough evidence to convince yourself and others that this time will be different**. **The unused request process is a paved road that leads nowhere people want to go**, with a goat track — the particular administrator — carrying the real traffic; sometimes the official way is the way everyone learns not to use. Learn the goat tracks or everything takes longer, and note that teams may ask you not to document them. **The quiet eleven-year engineer belongs to the shadow org chart**, and the remedy is to identify and convince them. (b) It is the classic signature of the **shadow org chart**: the unwritten structures through which power and influence flow, which are probably not the same as the actual org chart. It identifies two kinds of influencer — **connectors**, who know people right across the organisation, and **old-timers**, who wield influence regardless of rank or title simply from having been around a long time; people with rank and title trust them and rely on their judgement. The head of platform's first move in any decision is to check with this person, so **if they think it is a bad idea you will never get the head on board**. What a better document cannot do is convince them: **these are the people you have to convince before a change can happen**, and the way to find such lines of influence, which are not obvious when you join, is to make friends and ask how the organisation works. (c) **Timing:** the proposal landed two weeks after the half-year plans were signed off. If you announce an initiative immediately after the planning cycle has set objectives, **you will have a hard fight and may have to wait a quarter before seeing any progress** — which predicts exactly the silence being observed, without anyone having rejected the idea. **Channel:** posting broadly and waiting assumes the organisation makes decisions in the open, which is a question about the cultural attributes. Knowing whether the place is top-down or bottom-up tells you whether to take an idea to fellow grassroots practitioners and get their support first or to start by convincing your local director; knowing whether it runs on front doors or back channels tells you whether a broadcast is the accepted route or whether you needed an “in”. Broadcasting predicts visibility, not agreement. (d) The rewrite violates the principle that **being right about a need for change is less than half the battle**: you also have to convince other people that you are right and, harder, **convince them to care**. Forty more pages improve only the correctness half, which was never the constraint. A better order: first find and convince the influencer, because nothing moves until the head of platform's adviser is on board; then gather the evidence that distinguishes this attempt from the two abandoned ones, since without it the desert argument will be made for you; then obtain sponsorship or work out the password for the security fortress; then reintroduce the proposal in phase with the planning cycle rather than immediately after it; and use whichever channel this organisation actually treats as the way things get decided.

PART 2 · Q2 — THE RETAILER WITH NO DIRECTION

(a) **Harder to keep everyone going in the same direction** — teams are making incompatible local choices, so either they go to the wrong place or every decision becomes long, complicated and full of discussion. **You will not finish big things** — the two-year consolidation abandoned at eight months is the visible form; short-term projects that solve local pain points crowd out problems that take multiple steps, and the value of existing projects may not be clear to anyone outside the team. **You accumulate cruft** — the checkout abstraction and the fulfilment format are the two forms of it, treated in (b). **You get competing initiatives** — three engineers rallying enthusiasm around three different messaging platforms, each trying to do the right thing and get people aligned, with chaos as the end result. **Engineers stop growing** — the junior engineers can describe their tickets and not their purpose; focusing only on short-term goals limits how they think about and frame their work and how much ownership they take of what falls between tasks, and a team used to iterating on short, clearly specified goals **will not build the muscle for bigger, more difficult projects and will not be able to tell the story of why they did what they did**. (b) The category is **cruft**, and the mechanism has exactly two branches, which is why opposite behaviour produced the same damage. If teams do not know where they are going they can either try to be **flexible enough to support every future state**, creating solutions that are overcomplicated and hard to maintain — the checkout team's three payment models — or they can **make local decisions**, taking the risk that their direction will not match everyone else's and that their solution becomes **a weird edge case that everyone else has to work around** — the fulfilment team's format and the converters now written by every other team. (c) The diagnosis is wrong in kind. Stricter delivery reviews address execution, and none of the five symptoms is an execution failure; each one is what a missing destination produces. What the organisation lacks is a **treasure map**: a shared long-term destination with a clear definition of success. What having it changes is precise — **if everyone knows where they are going there is no need to keep tight alignment along the way**: each team can be more creative about its own route, with its own narrative for the problems it must solve; they are less likely to go down wrong paths; and they have enough information to make decisions, **reducing the hedging and technical debt they need to incur**. They can also celebrate the wins along the way while knowing the real celebration comes at the destination. If it turns out that no destination was ever chosen, or that there are several incompatible ones, the useful first act is to write up where you see the confusion or misalignment and share it, which forces the conversation — or the argument — into the open; and at that point the job stops being to uncover a map and becomes to create one. (d) The missing idea is the **technology tree**: capabilities form a directed graph, and much of what you invest in is an unavoidable step on the path to something else — you research physics because you want to build a railroad.

Faster service startup is a node, not a destination. The rewrite: *we are doing this so that new services can be set up quickly, so that teams can ship features faster; the real goal is to reduce time to market, and here is how we will know whether it moved.* That version can be evaluated, which is the point — **when you know the real goal you can step back and assess whether any proposed work will get you closer to it**, a test that cannot be applied to a project described only by what it builds. And the warning attached: **unless you remember where you intended to go and keep working on it, you never get to ride the train**.

PART 3 · Q1 — THE LOGISTICS SCHEDULING LEAD

(a) The stated test is that **if it is difficult to articulate the goals — or if the goals are just a description of your implementation — you have not spent enough time in exploration**. "Build a streaming scheduler on a distributed queue" is purely an implementation, so the document fails the test outright. The related rule from design documentation says the same thing from the other side: **the goal should not include implementation details**, because specifying the technology in the goal means a major design decision has been made without being justified, and **the specific implementation should serve the goal, not be the goal**. The skipped stage is **exploring**, which exists to work out the project's true needs and evaluate the approaches available. It should produce: the ability to **explain concisely what different teams want in a way they will agree is accurate**; a crisp definition of what you are doing, reducible to an elevator pitch; and **a clear description of what you are not doing**, including how neighbouring projects overlap with or sit inside yours. (b) The hazard is that teams **may have agreed to help because they think your project will also achieve some other goal they care about — and they might be wrong**. It belongs to the same family as teams holding different mental models of what is being achieved, having unspoken constraints, fixating on niche use cases, or using the same vocabulary to mean different things. The lead should surface it during exploration rather than at delivery: talk to that team and listen to what they actually say and the exact words they use rather than mentally filling in what you wish they had said; get to the point of being able to state their want in a way they agree is accurate; and then **state explicitly what the project is not doing**, so the mistaken expectation is corrected while correcting it is still cheap. (c) The missing structure is **defined roles** — at its simplest a table of leadership responsibilities against who owns each, and more formally **RACI**: **Responsible**, the person actually doing the work; **Accountable**, the person ultimately delivering it and signing off that it is done, of whom **there is supposed to be only one per task**, often the same person as the responsible one; **Consulted**, people asked for their opinion; and **Informed**, people kept up to date on progress. The beginning of a project is the best time to lay this out, rather than waiting until two people discover they are doing the same job or work slips through the cracks because nobody thinks it is theirs. The two patterns it exists to break are **never making a decision because nobody knows who the decider is, so the group discusses forever**, and **relitigating every decision over and over because there is no process for making decisions**. Its real superpower is that it is an **uncontroversial structure**, which gives you a way to broach the conversation without it being weird. Note also that with no product manager, someone must own user-visible behaviour and customer requirements — as lead you are **implicitly filling any role that does not already have someone in it**. (d) **Availability.** As the person responsible for moving the project along, your time is less predictable than other people's and you will have more meetings than anyone, so if you take on the biggest, most important problems you will take longer to get to the coding work than someone else would — **which blocks others and makes you a bottleneck**. If you are coding, pick work that is not time-sensitive or on the critical path; better, think of your code as a lever that helps everyone else, pair rather than take over, and avoid becoming a single point of failure. **The measure.** Percentage of the engine completed reports the lead's own output. **The existence of code tells you nothing about whether any problem has actually been solved** — in some cases success comes from deleting code — so this cannot show progress toward anything. Success metrics should be agreed with the sponsor and the other leads, and status should be explained **in terms of impact**: what users can do now and when they will be able to do the next thing.

PART 3 · Q2 — THE PERMISSIONS DESIGN DOCUMENT

(a) **It's a brand-new problem (but it isn't).** There are occasional exceptions, but your problem is almost certainly not new; the missing alternatives section is where you demonstrate to yourself and others that you are here to solve the problem and are not just excited about the solution, and **omitting it because you considered no alternatives is a signal that you may not have thought the problem through**. Why would simpler solutions or off-the-shelf products not work? **Building for the distant, distant future.** If you are designing for orders of magnitude beyond current usage you need a real reason; watch out for overengineered solutions much more complicated than they need to be, and if you are adding automatic region failover, explain why it is worth the extra time and effort — **"we might need it later" is not a good enough justification**. **Every user just needs to…** With five users you can individually teach each of them all the arcane rules of your system; with hundreds **they are going to do it wrong, and if you do not plan for that your design does not work**. Any part of a solution that involves humans changing their workflows or behaviour will be difficult and **needs to be part of the design**. **We'll figure out the difficult part later.** This is common in migrations: you spend a quarter building and polishing the system for a couple of easy cases and then have to work out how to make it work for the difficult ones — **and what happens if that turns out not to be possible?** Ignoring the difficult part can also mean pushing complexity onto someone else, such as requiring every existing caller to change rather than being backward compatible. (b) A rewrite: *"The permissions service will validate the caller's role against the requested resource before it returns a decision. The client library will refresh the caller's access token when that token is within five minutes of expiry."* The first technique is to **be clear about who or what is doing the action for every single verb** — writing in the passive voice, as in "the data will be encrypted in transit", obscures information and makes the reader guess, so use active verbs with a subject who does the action. The

second is that **it is fine to use a few extra words or even repeat yourself if it means avoiding ambiguity**: instead of a bare “this” or “that”, add a noun spelling out exactly what you are referring to, even if you have just mentioned it — here, saying *which* token is refreshed and against what the access is validated. (c) The principle is **wrong is better than vague**. People are hand-wavy, or avoid committing to a plan at all, precisely because they do not want others to argue with them about the details — and it works, which is the problem. **If you are wrong, people will tell you and you will learn something, and you can change direction if you need to**; if you are trying out a controversial idea, you find out early whether your colleagues will pull against your approach. Having disagreements about your design does not mean you need to change course, **but it gives you information you would not have had otherwise**. Vagueness purchases the absence of argument at the cost of the absence of correction — and since **a written design is a very cheap iteration**, what is being declined is the cheapest correction available. (d) The meeting was consumed by **bikeshedding**, from the **1957 law of triviality**, which holds that **since it is much easier to discuss a trivial issue than a difficult one, that is where teams tend to spend their time**; the original example was a fictional committee evaluating the plans for a nuclear power plant that spent the majority of its time on the easiest topic to grasp, the materials for the staff bicycle shed. Even senior people drift into long paragraphs on the most trivial, reversible decisions while not engaging at all with the ones that are harder to grasp or to find consensus on. The two sections that were required regardless of the author's belief are **security, privacy and compliance** — **even if you think there are no security concerns and the section is not relevant, write down why you believe that is the case** — and **alternatives considered / prior art**. Both belong to the small set of headings described as the ones that absolutely have to be included even in a tiny document, because they are **the “keep you honest” sections**.

PART 4 · Q1 — THE SENIOR ENGINEER'S FIRST QUARTER

(a) **Merging without review.** This is **values are what you do**: no matter what the written principles say, if senior engineers approve changes without reading them, **everyone else will rubber-stamp code reviews too** — the work you do is implicitly the type and standard of work that others will see as correct and emulate. The predicted effect is precisely what has been observed, and note that nobody had to be told. **The confident answer.** Bluffing. The prediction is that junior people conclude not knowing is unacceptable, so they bluff or quietly disengage rather than be caught out — and **it slows everything down when people in a conversation bluff or disengage**, because we spend a huge amount of our working lives trying to get a shared model of the world into our brains so that we can collaborate and make decisions. **The outage response.** This is **cover-your-ass engineering** — “it's not my fault, they broke it, they used it wrong, I built it to spec” — and it predicts that nobody else on the team will be open about their own mistakes either, because a leader being open about theirs is what **makes it easier for everyone else to do the same** and is a big boost to the team's psychological safety. It also breaks a specific obligation: if there is any risk that someone else will get blamed, you clarify that they did nothing wrong, and this did the reverse. **Praising the glue work.** The prediction is not about culture but about a career: the newest engineer is spending their prime technical learning years on work that does not teach them technical skills, and leaders often do not step in because the work is needed for the project to succeed **and they are just glad it is getting done**. (b) Beyond being wrong, bluffing costs **the opportunity to learn** — and it may lead to bad decisions — and it **wastes the opportunity to set an example**: every time you admit you do not know everything and let people see you learning, you show junior engineers that it is normal to keep learning. It is specifically safer for this person to ask because **technology can be fraught with egos and insecurity, and it is sometimes scary — or legitimately risky — for junior people to admit they do not know something**; the corollary is that **if senior people can admit they do not know things, everyone else will do it too**. (c) The response should satisfy three obligations. **Admit what happened and then set out to fix it** — do not beat yourself up, but do not deny the impact or insist the mistake was not really your fault. **Communicate quickly and clearly so everyone has the information they need**, and because a colleague is at risk of carrying it, state explicitly that the approving engineer did nothing wrong. **Hold a retrospective** — talk through what happened, how you recovered and what you learned, being open and matter-of-fact about the part you played, since **it is much easier to understand what happened when nobody is trying to downplay their missteps**. Solving the problem you caused may be the last thing you want to do in that moment and it is the best thing you can do for the goodwill of your team. (d) The work is **glue work**: the leadership and administrative tasks that are not on anyone's job ladder but are needed to make a team successful — unblocking, onboarding, reminders, mentoring, scheduling. It is a hazard rather than a success for three stacked reasons: it is **rarely rewarded or allocated fairly** and falls to whoever cannot look away from the problem, often a junior person with a strong sense of ownership; when junior people do too much of it and not enough technical work they **spend their prime technical learning years in a way that does not teach them technical skills, which can stunt their careers in the long run**; and the same work is read differently depending on who does it — managers, promotion committees and future employers **may consider it leadership when a staff engineer does it but dismiss it when a more junior engineer does**. What the senior engineer should do is invert their instinct: recognise the work and understand who is doing it, **take ownership and do a lot of the work that is not anybody's job but that furthers the goals**, and **redirect their junior colleagues toward tasks that will develop their careers instead**. Meeting notes are the standard illustration — a junior person taking them cannot participate and it reads as low-status administration, whereas a senior person taking them is seen to be making the meeting effective, and the notes are a lever besides, letting you record the facts you think matter, document decisions and frame them first.

PART 4 · Q2 — THE SCHEDULING SERVICE

(a) The two principles are **keep it simple** — part of optimising for maintenance rather than creation — and **build to decommission**. On the first: senior engineers sometimes think they can demonstrate their prowess with the flashiest, most complicated solutions, but **it is easier to make something complicated and much harder to make it simple**, and one should beware organisations that seem to reward complexity, where people create complicated solutions to prove that they are doing hard work. What we should want are **simple solutions to complex problems — the complexity of our work is a cost to bear, not something to maximise**. The concrete method: when you think of a solution, **treat it as "just the first" and spend at least the same amount of time on another**, and now that you understand the problem better see if you can make it simpler — fewer lines of code, fewer branches, fewer teams, fewer hours of maintenance, fewer running components, fewer files touched; **the longer the system is intended to last, the longer you should spend**. Where complexity is genuinely inherent, make a deliberate decision about **where in the system to put it**, so that someone looking at the whole can treat that component as a black box and reason about everything else. (b) The hard-coded calls will cost whoever has to replace or remove the service, and the practice that answers it is **building to decommission**. Your architecture will evolve and your components will settle into the middle; while it might be faster now to wire the new thing straight in, ask **whether it will be possible to replace it later without demolishing whatever other people have built on top of it**. The test: **imagine knowing that you personally will need to decommission this component in ten years, and that Future You will be no less busy than Present You** — so add a clean interface, make it easy to see which clients are still using it, and design in a way that keeps a little distance between two systems being integrated. The side effect is the payoff: **if you set out from the start to build a component that is easy to decommission, you will have built something modular and easy to maintain**. (c) There are two options for letting future people understand a system. One is to **focus on education and hands-on experience** — running continual classes so that everyone who might have to work on it is fully trained and has logged enough hours to handle problems. The other is to **make it as easy as possible for people to understand it when they need to**: documentation written with that future person as the main audience — a clear, short introduction; at least one big, simple picture, using arrows to show which direction data moves; links to anything they might wonder about — and then exposing the system's inner workings as clearly as possible through tooling, tracing or useful status messages, so that it is easy to inspect, analyse and debug. By offering to explain it to anyone who asks, this team has implicitly chosen the first, and the reason that fails is that **the system will never again be as well understood as it is on the day it is created**, with that knowledge decaying a little every day. On the backups the position is unambiguous: **if you have not tested restoring your backups, assume you do not have any**. (d) **The justification.** Technical impressiveness is not a reason. You are working for an organisation that has goals, and **the software is the means to those ends, not an end in itself**; keep the idea of a budget in the back of your mind, and know how the company makes money and whether these are good times or lean ones. More sharply, **headcount is finite and staffing a project has an opportunity cost**, and you should be judicious about where you spend your **innovation tokens** — the organisation's limited capacity to do something creative, or weird, or hard. **Build the most useful thing, not the thing that would be most fun to build**, and when it is time to stop polishing something and declare it good enough, stop. In a freeze the justification has to be expressed in what the business needs. **The behaviour during the incident.** The relevant attribute is **creating calm**: if you are dealing with a big problem try to make it smaller, and if it is a small one keep it small. **A senior person making a fuss can blow up a minor thing into a big, loud issue**, so be certain you have the facts and that joining in genuinely helps — if your actions would amplify rather than calm, consider staying out of it. And be cautious about where you share your anxieties: **do not let your worries spill out on more junior people — it is not fair to ask them to carry your concerns, and you are amplifying the problem if you upset them**. Take the uncertainty instead to a manager, a close peer or a chosen sounding board, being clear whether you are asking for action or unpacking something for yourself, since otherwise the person may reflexively react and amplify what you meant only as venting.

Spaced-review tracker

Review at **day 1, 3, 7, 16, 35**. Write the date in the box when you pass. On a fail, reset that row to a 1-day interval and start again.

ITEM	DAY 1	DAY 3	DAY 7	DAY 16	DAY 35	FAILS
1 · The three maps, and why they are obscured not absent						
2 · Skill versus opportunity in knowing things						
3 · The four risks of depth, and which two are mirrors						
4 · The outsider view, and what it is not a licence for						
5 · The objectives that are always true						
6 · Measuring from the customer's point of view						
7 · The test for whether your work wastes your time						
8 · Being right is less than half the battle — and planning cycles						
9 · The seven cultural attributes						
10 · Power, rules or mission — the three types across seven rows						
11 · Chasms and fortresses, and their different remedies						
12 · Disputed territory, deserts, paved roads and goat tracks						
13 · The room where it happens, and the cost of admitting you						
14 · The shadow org chart — connectors and old-timers						
15 · Bridging, and why the problems are human ones						
16 · The five costs of thinking only short-term						
17 · The two branches by which cruft accumulates						
18 · The technology tree, and the real goal						
19 · Why big projects are hard, and what you own anyway						
20 · The five things that reduce overwhelm						
21 · The eight points of context						
22 · Why code written is not a success metric						
23 · RACI, and the two patterns it breaks						
24 · The triangle, and milestones as beta tests						
25 · Estimation, and telling dependent teams early						
26 · The test for whether you have explored enough						
27 · The five minimum design headings, and the two honest ones						
28 · Wrong is better than vague, and the two precision techniques						
29 · The ten design pitfalls						
30 · Status in impact, and the watermelon						

31 · Passive influence, and values being what you do						
32 · The four attributes, and what competence is built from						
33 · Why admitting ignorance is a senior person's job						
34 · Owning mistakes — the three obligations						
35 · Radiating intent, and what it keeps						
36 · Glue work, and why it is read differently by rank						
37 · Taking charge explicitly, and creating calm						
38 · Remembering the business, the budget, the user and the team						
39 · Maintenance over creation, and the method for simplicity						
40 · Build to decommission, and its side effect						

Final quiz — no notes, twenty minutes, write the answers

1. Name the four risks that come with depth in a domain, say which two are mirror images and what single cause produces both, and give the test for whether your current work is worth your time.

2. Explain why paying attention is not sufficient for knowing what is going on, name the second requirement, and give three of the six standing ways to keep an organisational picture current.

3. List the seven attributes on which organisational personality is described, and give one example of a single behaviour that is virtuous at one end of an attribute and disqualifying at the other.

4. Name the five terrain features that obstruct or assist a route, give the distinct remedy each takes, and say what most gatekeepers are actually motivated by.

5. Give the five consequences of a group having no long-term destination, and set out both branches of the mechanism by which such a group ends up with unmaintainable systems.

6. List the eight things you must find out before a large project can be said to have started, say which of them can legitimately end the project, and explain why counting produced code is a defective measure of progress.

7. Give the four responsibilities in the assignment matrix with the numerical constraint attached to one of them, name the two dysfunctions it is meant to remove, and say what its real advantage is.

8. State how you can tell that a team has not investigated its problem enough, list the headings no design document may omit, and give both techniques for making prose unambiguous.

9. Name the four qualities a senior engineer models involuntarily, say what most reliably reveals what an organisation truly rewards, and explain why declaring ignorance is more valuable coming from an experienced person.

10. Explain the two advantages of announcing an action in advance over requesting approval for it, describe why unassigned coordination work damages the career of whoever absorbs it, and give the exercise that produces a component someone can later switch off.